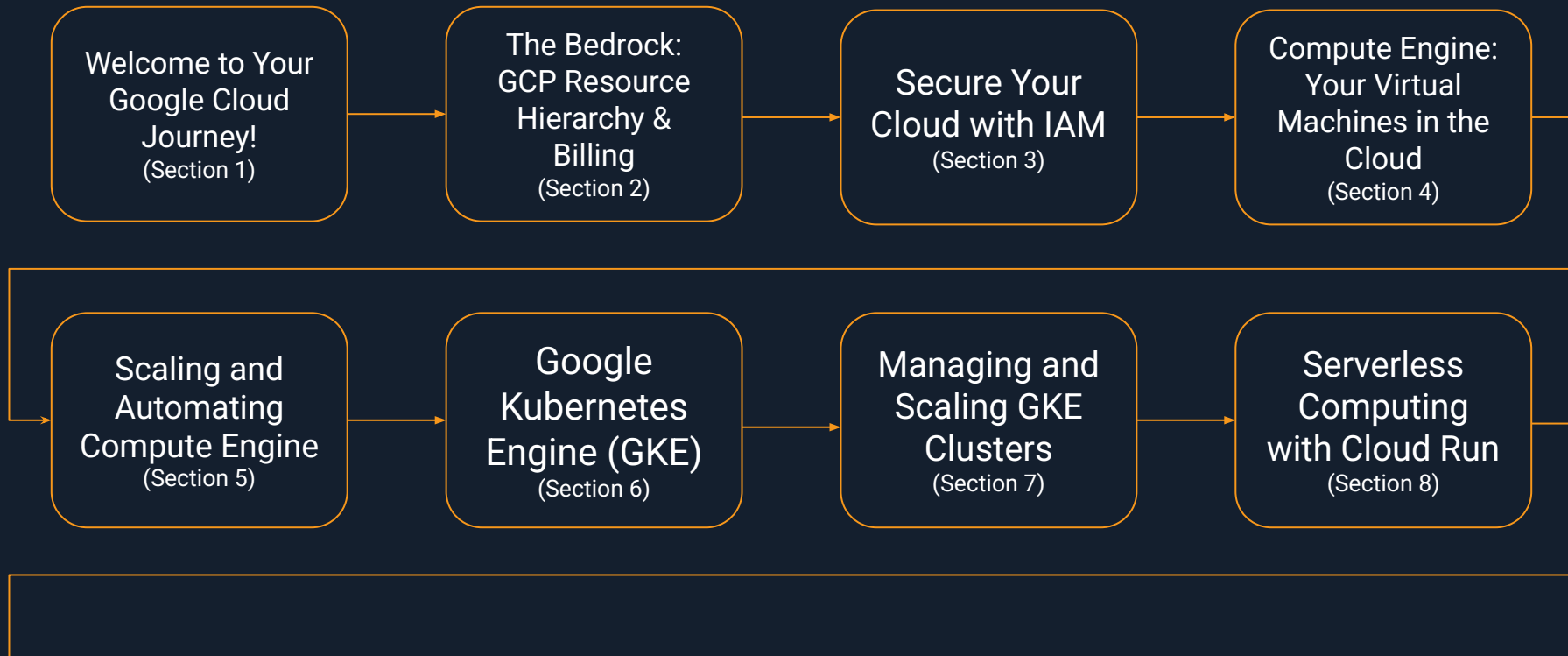


Section 1: Welcome to Your Google Cloud Journey

Become Google Cloud Associate Cloud Engineer Certified

by Vladimir Raykov



**Cloud Storage:
Your Infinitely
Scalable Object
Store**
(Section 9)

**Managed
Databases in
GCP**
(Section 10)

**Big Data &
Analytics
Platforms**
(Section 11)

**Managed File
Storage**
(Section 12)

**Virtual Private
Cloud (VPC)
Networking
Fundamentals**
(Section 13)

**Connecting
Your
Networks**
(Section 14)

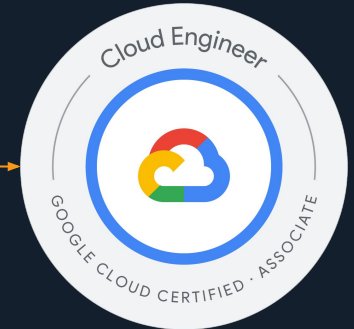
**Load Balancing
and Network
Tiers**
(Section 15)

**Infrastructure
as Code (IaC)
with Terraform**
(Section 16)

**Monitoring with
Cloud
Monitoring**
(Section 17)

**Logging and
Diagnostics**
(Section 18)

**Full-Length
Practice Exams**
(Section 19)



What is Cloud Computing & Why Google Cloud Platform (GCP)?

1. **Cloud Computing** is the on-demand delivery of IT resources over the internet, primarily using *pay-as-you-go* pricing, though other pricing models like **committed use discounts** are also available.
2. The cloud model helps businesses shift their spending from **Capital Expenditure (CapEx)** to **Operational Expenditure (OpEx)**, providing financial flexibility and reducing large upfront costs.
3. Google Cloud's key differentiators are
 - a. its high-performance global network with Premium Tier routing,
 - b. its world-class data analytics and AI services, and
 - c. its leadership in the open-source community with technologies like **Kubernetes**.
4. An **Associate Cloud Engineer** is responsible for **deploying, securing, monitoring, and maintaining applications and infrastructure** on Google Cloud.

The Big Picture: GCP's Global Infrastructure

1. **Regions** are large, independent geographical locations.
 - a. They are the building blocks for **disaster recovery (DR)**.
 - b. A failure in one region does not affect another.
2. **Zones** are isolated deployment areas within a region, representing *physically separated* infrastructure.
 - a. They are the building blocks for **high availability (HA)**.
 - b. Deploying your application across multiple zones in a region protects it from a single data center failure.
3. Resources are usually deployed to a **specific zone** (e.g., us-central1-a).
4. **Edge Locations (PoPs)** are points all over the world that get user traffic onto Google's fast private network as quickly as possible, reducing latency for services like **Cloud CDN**.

A Guided Tour of the Google Cloud Console

1. First, always be aware of the **Project** you are working in using the **Project Picker** at the top of the page.
2. Second, the **Navigation Menu** on the left lets you browse all services.
 - a. Star your **favorites** to keep them at the top for **quick access**.
3. Third, the **Search Bar** is the **fastest and most efficient way** to find any resource or service in Google Cloud.
4. And finally, the **Cloud Shell** gives you instant **command-line access** to your environment directly from your browser.

GCP's Footprint: Checking Service Availability by Region

1. The official, most accurate source for service availability is the **Google Cloud Locations page** at <https://cloud.google.com/about/locations>
 - a. Bookmark this page.
2. You can use the **continent tabs** to search for a **product's availability across different regions**.
3. **When creating a resource** in the GCP Console, the **Region** and **Zone dropdowns** are context-aware and will only show you the valid locations for that specific service.

Your Three Ways to Interact with GCP

1. **The Google Cloud Console** is your web-based Graphical User Interface (GUI).
 - a. *Use it for visualizing, discovering, and performing simple, one-off tasks.*
2. **The Cloud Shell** is your command line in the cloud.
 - a. *It's the primary tool we'll use in this course because it's easy, pre-configured, and consistent.*
3. **The Cloud SDK** is the command-line toolset you install on your own computer.
 - a. *Use it when you need to integrate GCP management with your local development environment.*

Section 2: The Bedrock: GCP Resource Hierarchy & Billing

The GCP Resource Hierarchy: Organizing Your Digital Assets

1. **The GCP Resource Hierarchy** has four levels — **Organization**, **Folders**, **Projects**, and **Resources**.
2. The **Organization** is the root of everything. **Folders** group projects, often by department.
3. **Projects** are where resources live, billing is managed, and APIs are enabled.
4. And the most important concept:
 - a. **Policy Inheritance** — *permissions applied at a higher level automatically flow down to everything below.*

Hands-On: Creating and Managing Projects

1. **Project Picker:** Click the project name at the top of the console to switch between projects—all resources and actions are scoped to the currently selected project.
2. **Project Name vs. Project ID:** Project Name is changeable; Project ID is globally unique and **permanent** (cannot be modified after creation).
3. **Project Location:** Projects can be created under an Organization or inside Folders; personal accounts show "No organization".
4. **Project Shutdown:** Use IAM & Admin > Manage Resources to shut down projects—30-day recovery period before permanent deletion.

Controlling Your Environment with Organization Policies

1. **Organization Policies** are the rulebook for your cloud.
 - a. They set guardrails on **what can be configured**, **NOT** who can do it.
2. The "who" is managed by IAM (Identity and Access Management).
 - a. This distinction is critical.
3. Policies are built by applying Constraints, which are predefined rules provided by Google for various services.
4. Organization Policies are **inherited** down the resource hierarchy.
 - a. A policy set on a Folder applies to all Projects within it, providing a powerful way to enforce rules at scale.

Hands-On: Applying Organization Policies to Projects

1. First, we confirmed that an **Organization is required** to manage these policies.
2. We learned that to edit policies, you need the correct IAM permissions, and we walked through the exact steps to grant the **Organization Policy Administrator** role.
3. We navigated to the Organization Policies page and selected the **Resource Location Restriction** policy.
4. We **defined a custom policy** to only allow a specific list of GCP locations.
5. And most importantly, we **tested our policy** and confirmed that it proactively filtered the available options in the console, enforcing our rule.

Hands-On: Activating the Tools: Enabling APIs for Your Projects

1. Every service in Google Cloud is controlled by an **API**, which acts as the "on switch" for that service within a project.
2. The **API Library** is the central catalog where you can discover and enable any GCP service API.
3. Before you can create resources like Virtual Machines or Databases, the corresponding **API must be enabled** for your project.
4. *A common reason for errors when running scripts or applications is forgetting to enable a required API, so it's one of the first things you should check when troubleshooting.*

Hands-On: Creating Your First User and Group in Cloud Identity

1. **Cloud Identity manages identities**—the "**who**"—in your Google Cloud organization.
 - a. *It is separate from IAM, which manages permissions—the "what you can do."*
2. You can manage individual **users**, but **the industry best practice is to manage permissions using groups**.
3. **Groups make managing access scalable and secure**.
 - a. *You assign permissions to a group, and then simply add or remove users from that group.*
4. You manage users and groups in the **Google Admin Console**, using the direct "Manage Users" and "Manage Groups" buttons on the "Identity & Organization" page in GCP.

Hands-On: Day-to-Day Management of Users and Groups

1. For users, you can easily **Reset Passwords**, **Suspend accounts** for temporary leaves, and securely **Delete accounts** while transferring their data for permanent departures.
2. For groups, you can **add new members at any time** to grant them access, and you can delegate responsibility by promoting members to a **Manager role** within the group.
3. All of these common lifecycle tasks are handled efficiently within the **Google Admin Console**.

Hands-On: Discovering Your Cloud Assets with Asset Inventory and Gemini

1. First, the **Cloud Asset Inventory** is *the central catalog for every single resource in your Google Cloud organization.*
2. Second, you can use the UI on the **RESOURCE tab** *to filter and search for specific asset types to quickly find what you're looking for.*
3. And finally, you can use **Gemini** as an expert assistant.
 - a. When you're unsure how to do something, you can ask it for help, and it will provide you with clear, step-by-step guidance to accomplish your task.

Billing 101: How GCP Billing Works

1. A **Cloud Billing Account** is your central payment profile with Google.
 - a. It's where you define your payment method.
2. **Billing Accounts and Projects are separate resources.**
 - a. *This allows for flexible and centralized financial management.*
3. For a project to use paid services, **it must be linked** to an **active** billing account.
 - a. *This link is what tells Google where to send the bill.*
4. All costs are tracked on a **per-project basis**, giving you a detailed breakdown of your spending.
5. We can use **labels** to tag our resources for detailed cost attribution, which helps us understand our spending with much greater clarity.

Hands-On: Verifying and Managing Project Billing

1. **Automatic Billing Linkage:** Projects automatically link to a billing account if you have access to only one; multiple accounts require manual selection during project creation.
2. **Account Management Page:** Navigate to Billing > Account Management to view all projects linked to a billing account.
3. **Change Billing:** Use the three-dot Actions menu to reassign a project to a different billing account (requires appropriate IAM permissions).
4. **Billing Verification:** Always verify project-to-billing-account linkage after project creation, especially in organizational environments.

Never Get a Surprise Bill! Setting Up Budgets and Alerts

1. **Proactive, Not Reactive:** *Budgets are essential tools that let you monitor your spending proactively, so you can act before the bill arrives.*
2. **Key Components:** *A budget is defined by its **scope** (what it covers), its **amount** (the target), and its alert **thresholds** (the triggers).*
3. **Alerts, Not Actions:** *It's critical to remember that budgets only send notifications.
 - a. They do NOT automatically **stop** services or cap your spending.*
4. **Automation is Possible:** *While the default is email alerts, you can use programmatic notifications with Pub/Sub and Cloud Run Functions to build powerful, automated responses to budget alerts.*

Hands-On: Creating Your First Billing Budget

1. **Budget Creation Path:** Navigate to *Billing > Budgets & Alerts* to create and manage budgets.
2. **Three-Step Process:** **Scope** (*what/when it applies*), **Amount** (*target value*), **Actions** (*alert thresholds and notifications*).
3. **Default Alert Thresholds:** *Budgets automatically include 50%, 90%, and 100% threshold alerts based on actual spend.*
4. **Default Notification Recipients:** *Alerts are sent to billing admins and users with the Billing Account Costs Manager role by default.*

Hands-On: For the Accountants: Setting Up Billing Exports to BigQuery

1. **BigQuery Dataset Required:** *Create a BigQuery dataset first to serve as the destination for billing export data.*
2. **Billing Export Location:** *Navigate to **Billing > Billing Export** to configure exports.*
 - a. *“Detailed usage cost” provides resource-level granularity*
3. **Export is Forward-Looking:** *Billing exports start from the moment of configuration—no historical data is included, and initial data population takes several hours.*
4. **Table Auto-Creation:** *Google automatically creates a billing export table in your designated dataset*

Understanding Resource Quotas and How to Request Increases

1. **Quotas are Safety Limits:** *They exist to protect both you from overspending and the platform from being overwhelmed.*
2. **Two Main Types:**
 - a. Allocation quotas *limit the amount of resources (like CPUs), and*
 - b. Rate quotas *limit the frequency of actions (like API calls).*
3. **Central Management:** *The 'IAM & Admin' > 'Quotas & System Limits' page is your single source of truth for viewing all project quotas.*
4. **Filtering is Key:** *While you can often see what you need right away, the filter bar is powerful for finding specific quotas in large projects.*
5. **Requesting Increases:** *The process is straightforward, but always provide a clear and valid justification for your request.*

Section 3: Secure Your Cloud with Identity and Access Management (IAM)

IAM Explained: Principals, Roles, and Policies

1. **Principal (The 'Who'):** *This is the identity requesting access.*
 - a. It can be a user, a group, or a service account for an application.
2. **Role (The 'What'):** This is a collection of permissions that define what actions can be taken.
 - a. It's a job function, like '**Viewer**' or '**Editor**'.
3. **Policy (The Binding):** This is an allow policy made of role bindings.
 - a. *A binding connects principals to a single role on a resource.*
 - b. **Policies are attached to resources** in the hierarchy and permissions are inherited downwards.

IAM Role Types: Basic, Predefined, and Custom Roles

1. **Basic Roles (Owner, Editor, Viewer):** These are the legacy, primitive roles.
 - a. *They are far too broad and should be avoided in production because they violate the **Principle of Least Privilege**.*
2. **Predefined Roles:** This is the recommended best practice.
 - a. *They are granular, service-specific roles **managed by Google** that allow you to give principals only the permissions they need for their job.*
3. **Custom Roles:** These offer the **most control**, allowing you to create your own roles with a specific list of permissions when predefined roles don't quite fit.

Hands-On: Granting and Revoking IAM Roles in the Console

1. **Granted Access:** We assigned a specific, predefined role (**Compute Viewer**) to a new user.
2. **Verified Access:** We logged in as that user to confirm they could see resources but couldn't perform any actions, and that they were correctly blocked from other services.
3. **Edited & Revoked:** We demonstrated how to change a role, saw the danger of the broad '**Editor**' role, and then completely removed the user's access.

Hands-On: Creating and Using a Custom IAM Role

1. **Created a Custom Role:** We defined a new role with a specific **title**, **ID**, and **description**.
2. **Found and Added Permissions:** We used the **service.resource.verb** pattern, the filter, and the correct workflow of searching, selecting, and clearing to add the four exact permissions our operator needed.
3. **Assigned and Verified:** We assigned our new role to a user and then logged in as them to prove, step-by-step, that they could do exactly what we intended and were blocked from doing anything else.

The Principle of Least Privilege: A Core Security Concept

1. **A Fundamental Rule:** *The Principle of Least Privilege means granting only the absolute minimum permissions necessary for a user or service to do its job.*
2. **Limit the Blast Radius:** *Its primary purpose is to dramatically reduce the potential damage from both **accidents** and **malicious attacks**.*
3. **Our GCP Toolkit:** *We implement it by avoiding Basic roles and using a combination of specific Predefined and highly-tailored Custom roles.*
4. **Crucial for Automation:** *The principle is even more critical for applications and **service accounts** than it is for humans.*

Your Applications' Identity: Understanding Service Accounts

1. **An Identity for Code:** *A service account is a non-human principal that gives your applications an identity to interact with Google Cloud services.*
2. **Two Main Uses:** *They provide an identity for resources running inside GCP and are used with downloadable keys to authenticate from outside GCP.*
3. **Dual Nature:** *A service account is both an identity that gets permissions and a resource that you can grant permissions on, like the critical 'Service Account User' role.*
4. **Managed vs. Default:** *It's a best practice to create your own user-managed service accounts with least-privilege permissions rather than relying on the broad, Google-managed default accounts.*

Hands-On: Creating and Assigning Service Accounts

1. **Created a Service Account:** We defined a new, user-managed service account with a clear name and description.
2. **Granted a Granular Role:** We assigned it the '*Storage Object Creator*' role, giving it only the permission to create files and nothing more.
3. **Assigned to a Resource:** We attached our new service account to a VM during creation, making sure to avoid the default service account.
4. **Verified from Within:** We connected to the VM over SSH and used **gcloud** and **gsutil** to prove that it could perform its allowed action and was correctly denied from performing any others.

Securing Service Accounts: Managing Keys and Permissions

1. **Service account keys** are only necessary when your application runs outside of Google Cloud and needs to authenticate to GCP services.
2. You should always avoid using service account keys if your code runs on GCP—use the built-in attached identity instead because it's more secure.
3. **A leaked service account key** *gives an attacker all the permissions of that service account*, making it a major security risk that requires strict least privilege application.
4. If you must use a service account key, you need to:
 - a. **rotate** it regularly,
 - b. **store** it in a secret manager, and
 - c. **monitor** its activity through audit logs.

Advanced IAM: Service Account Impersonation Explained

1. **Service Account Impersonation** *eliminates the need to download, secure, and rotate dangerous long-lived JSON keys while still granting the necessary permissions.*
2. A principal with the correct permission can temporarily act as a service account and receive short-lived credentials that expire automatically.
3. Impersonation is enabled by granting the **iam.serviceAccountTokenCreator** role to a principal on the target service account they need to impersonate.
4. The primary benefits are drastically improved security through eliminating static keys and providing clear audit trails that show exactly who performed which actions.

Hands-On: Configuring Service Account Impersonation

1. The '**Principals with access**' tab (IAM & Admin -> Service Accounts -> Click The Service Account) is *where you grant users permission to impersonate a specific service account-this is more granular than project-level IAM.*
2. The **roles/iam.serviceAccountTokenCreator** role is what enables impersonation-this role allows a principal to generate short-lived OAuth2 tokens for that service account.
3. Use **--impersonate-service-account=** flag in **gcloud** and **gsutil** commands to execute actions as a different service account **without** needing to download keys.
4. **Impersonation is tightly controlled by IAM** - you can only impersonate service accounts where you've been explicitly granted the Token Creator role, making it secure and following least privilege.

Best Practices for Managing Service Accounts Securely

1. **Create with Purpose:** Avoid default accounts. Use a strict naming convention and create one service account per workload.
2. **Grant with Caution:** Apply the Principle of Least Privilege and use impersonation as your default, avoiding static keys whenever possible.
3. **Monitor with Vigilance:** Use tools like **IAM Recommender** and **Cloud Audit Logs** to find **unused** permissions and alert on sensitive actions.
4. **Decommission with Confidence:** Follow a disable-then-delete lifecycle to safely retire service accounts that are no longer needed.

Section 4: Compute Engine: Your Virtual Machines in the Cloud

What is Compute Engine? Your First Virtual Machine (VM)

1. **Compute Engine** is Google Cloud's Infrastructure as a Service (IaaS) offering.
 - a. *It allows you to create and run Virtual Machines (VMs) on Google's global infrastructure.*
2. A VM is a **virtualized computer** with its own operating system, vCPU, memory, and storage.
3. You choose Compute Engine when you need **maximum control and flexibility** over the operating system and software environment.
4. **Key components of a VM are:**
 - a. its machine type (CPU/memory)
 - b. its boot disk (a Persistent Disk)
 - c. its network configuration
5. **Spot VMs** provide massive cost savings for fault-tolerant workloads that can be stopped and started.

Hands-On: Launching Your First Linux VM in GCP

1. **Compute Engine** offers both preset machine types for common workloads and custom types for precise *resource allocation and cost optimization*.
2. **Shared-core machine types** (like the E2 series) keep costs low by **sharing** physical CPU resources, making them ideal for small workloads.
3. A VM's operating system runs on a **boot disk**, which is a type of **Persistent Disk**; *these disks can be either **zonal** or **regional** resources*.
4. Firewall rules are required to allow specific types of traffic, such as **HTTP** and **HTTPS**, to reach your VM.
5. **Data protection** for a VM's disk can be configured using **snapshots** or the **Backup and DR** service, though Backup and (Disaster Recovery) DR is only available in select regions.

Hands-On: Launching a Windows Server VM in GCP

1. **Windows Server** is a licensed operating system, which significantly increases the cost of the VM compared to a **free** Linux distribution.
2. To launch a Windows VM, you must **change** the default Boot Disk from a Linux image to your desired Windows Server image.
3. Windows Server requires a larger default boot disk size, typically 50 GB, compared to the 10 GB default for many Linux images.
4. The standard protocol for managing a Windows Server with a graphical interface is **RDP (Remote Desktop Protocol)**, which **requires a firewall rule** to allow traffic on **TCP port 3389**.

Hands-On: Securely Connecting to Your VMs (SSH & RDP)

1. **SSH (Secure Shell)** is the standard protocol for connecting to and managing **Linux VMs** via the command line.
2. **RDP (Remote Desktop Protocol)** is the standard for connecting to a **Windows Server** with a graphical user interface.
3. *Google Cloud's in-browser SSH client is a highly secure method that automatically manages temporary SSH keys for you.*
4. To connect to a Windows VM, you must first set a **password for your user**, then download and open the **RDP file** to use a local client.
5. Access to connect to VMs using either SSH or RDP is fundamentally controlled by a **user's IAM roles and permissions**.

Choosing the Right Machine Type for Your Workload

1. Choosing the right machine type is a **critical balance** between performance and cost, a concept known as "**right-sizing**."
2. **General-Purpose machines** (like E2, N2) offer a balanced mix of vCPU and memory for common workloads like web servers.
3. **Compute-Optimized machines** (like C2, H3) are for CPU-intensive tasks like high-performance computing and gaming.
4. **Memory-Optimized machines** (like M3) provide large amounts of RAM for workloads like in-memory databases.
5. **Storage-Optimized machines** (like Z3) are for workloads needing high-performance local storage, like scale-out databases.
6. **GPU machines** (like A2, G2) are equipped with GPUs for specialized machine learning and AI tasks.
7. **Custom Machine Types** are a powerful feature that allows you to fine-tune the vCPU and memory, which is a key strategy for cost optimization.

Cost-Saving Strategy: Using Spot VMs

1. **Spot VMs** offer massive discounts (up to 91%) by using Google's **spare compute capacity**.
2. The main characteristic of a Spot VM is that **it can be preempted**—stopped or deleted—by Google at any time with a 30-second notice.
3. **They are ideal** for **fault-tolerant** and **stateless workloads** like batch processing, data analysis, and running containers on GKE.
4. **Never use Spot VMs for critical, stateful applications** like production databases or workloads that cannot handle interruptions.
5. **Spot VMs** are the successor to **Preemptible VMs** and have **no** maximum 24-hour runtime limit.
6. You can use Spot VMs within **Managed Instance Groups (MIGs)**, which will automatically try to recreate instances that have been preempted.

Hands-On: Launching and Testing a Spot VM

1. To create a Spot VM, you begin by clicking "Create Instance" just like for a standard VM.
2. The key configuration is located in the **Advanced section** of the VM creation page.
3. You must change the VM provisioning model from "Standard" to "**Spot**."
4. The On VM termination setting determines if the VM is stopped or deleted upon preemption; **"Stop" is the default and safer option.**
5. A stopped Spot VM can be restarted later, preserving its persistent disk.

Hands-On: Adding and Resizing a Persistent Disk

1. **Adding a new data disk is a multi-step process:**
 - 1.1. *create the disk,*
 - 1.2. *attach it to the VM*
 - 1.3. *format and mount it inside the OS*
2. You can **resize** a Persistent Disk while it is still attached to a running VM, which is a powerful feature for avoiding downtime.
3. **After resizing** a disk in the Google Cloud Console, you must also run a command like **resize2fs** inside the OS to **expand** the filesystem onto the new space.
4. It is important to identify the correct device name when mounting or resizing.

A Deeper Look at VM Storage: Persistent Disks & Hyperdisk

1. **Persistent Disks** are network-attached storage that exists independently from your **VM instances**.
2. You must choose the right disk type based on your workload: **Standard** for bulk storage, **Balanced** for general use, and **SSD** or **Extreme** for demanding databases.
3. **Hyperdisk** is the next-generation storage that **decouples** performance from capacity, allowing you to provision **IOPS** (Input/Output Operations Per Second) or **throughput** independently.
4. **Zonal disks** are confined to a single zone, while **Regional disks** replicate data across two zones for high availability.
5. *Choosing the right storage is a critical balancing act between your application's performance needs and your budget.*

Protecting Your Data: VM Snapshots and Images

1. **Snapshots** are point-in-time backups of a Persistent Disk.
2. Snapshots are **incremental**, meaning they only store the changes since the last snapshot, which saves time and money.
3. The primary use case for snapshots is backup, data protection, and disaster recovery.
4. **Images** are bootable disks used as blueprints to create new, pre-configured VM instances.
5. You use **Public Images** for clean OS installations and create **Custom Images** to replicate your own configured environments.
6. **Remember the core concept:**
 - a. **Snapshots** are for backing up data, while
 - b. **Images** are for creating new VMs.

Hands-On: Creating a Snapshot and Restoring a VM

1. **We created a file** *on our VM to represent a specific state of our data.*
2. **We created a snapshot** *of the VM's boot disk to backup that state.*
3. **We simulated a disaster** *by deleting the original VM instance entirely.*
4. **We created a brand new VM instance** *directly from the snapshot.*
5. **We verified** *that the new VM contained the exact data from when the snapshot was taken.*

Automate Your Backups: Scheduling Persistent Disk Snapshots

1. **Snapshot Schedules** are used to **automate the creation of Persistent Disk snapshots**.
2. A schedule defines the **backup frequency** (hourly, daily, or weekly) and **the start time**.
3. **A retention policy** *is a critical component that automatically deletes old snapshots to manage storage costs.*
4. **Snapshot schedules are regional resources** *and can only be attached to disks in the same region.*
5. **You must attach a snapshot schedule to a Persistent Disk for it to take effect.**
6. **One schedule can be attached to multiple disks**, including both boot disks and data disks.
7. **Automating snapshots is a best practice** *for ensuring data protection and compliance.*

Automating VM Management with VM Manager

1. **VM Manager** is a suite of tools for automating the management of large VM fleets.
2. It requires the **OS Config Agent** to be running on the managed VMs.
 - a. **Patch Management** automates OS patching and provides compliance reporting.
 - b. **OS policies** enforce a consistent software state on your VMs using declarative configurations.
3. Using VM Manager is a **best practice** for maintaining security and compliance at scale.

Centralized Access Control with OS Login

1. **OS Login** centrally manages Linux VM access using IAM roles instead of manual SSH key management.
 - a. *The Compute OS Login* role grants standard SSH access, while
 - b. *Compute OS Admin Login* role grants administrator (sudo) access.
2. It improves security by **automatically revoking access** when a user's Google account is disabled.
3. *OS Login can be configured to require users to have 2-step verification (2FA) on their Google account.*
4. **It simplifies auditing** because login events are tied directly to a user's Google identity.
5. Enabling OS Login **is the recommended best practice** for managing instance access at scale.

Section 5: Scaling and Automating Compute Engine

Don't Repeat Yourself - Using Instance Templates

1. **Instance Templates** are reusable VM configuration blueprints that save machine type, boot disk, network settings, metadata, and startup scripts for creating identical virtual machines.
2. **Templates are immutable** resources that cannot be edited after creation;
 - a. you must create a new template to make changes, ensuring configuration consistency.
3. Google Cloud offers both **regional** and **global Instance Templates**, *with regional templates recommended unless you need cross-region reusability.*
4. **Deterministic templates** explicitly specify software versions and dependencies, *ensuring VMs remain identical over time and preventing unexpected behavior.*
5. Instance Templates are **required** for creating **Managed Instance Groups (MIG)** and can be created from scratch, from existing VMs, or from other templates.
6. **Templates cost nothing to store**; *you only pay for actual VM resources created from them, and Google recommends referencing templates by ID rather than name for security.*

What are Managed Instance Groups (MIGs)?

1. **Managed Instance Groups (MIGs)** manage multiple identical VMs as a single entity using instance templates
2. MIGs provide **autoscaling**, **autohealing**, **regional deployment**, and **automatic updates**
3. Regional MIGs support up to 2,000 VMs across multiple zones, while zonal MIGs support 1,000 VMs in a single zone
4. **Regional MIGs are recommended** for **high availability** as they protect against zonal failures
5. There is no additional charge for using MIGs, only for the underlying VM resources

Hands-On: Creating an Autoscaled Managed Instance Group

1. **Managed Instance Groups** require an **Instance Template** to define the configuration of each VM in the group.
2. To create a scalable solution, you configure an **autoscaling policy** on the Managed Instance Group itself, not on the template.
3. Autoscaling is commonly based on metrics like **CPU utilization**, and **you must define minimum and maximum instance counts** to control size and cost.
4. **Regional MIGs** provide high availability by distributing instances across multiple zones within a single region.
5. **Autohealing policies** use **health checks** with an appropriate initial delay to detect and automatically recreate failed instances, ensuring application uptime.
6. **Startup scripts** are a powerful tool within an **Instance Template** to automate the installation and configuration of software on new VMs.

Understanding Autoscaling Policies: Scaling on Metrics

1. **Autoscaling** is not limited to just CPU.
 - a. You can scale based on Load Balancer requests, Cloud Monitoring metrics, or a fixed schedule.
2. Scaling on **Load Balancer utilization (%)** is ideal for web applications where backend serving capacity is the best measure of load.
3. **Cloud Monitoring** metrics provide the most flexibility, allowing you to scale on standard metrics like memory (requires Ops Agent) or custom application-level metrics.
4. **Schedule-based autoscaling** is used for predictable, time-based traffic patterns, allowing you to set minimum instance counts for specific times.
5. **Predictive autoscaling** uses machine learning to forecast load and scale proactively to meet demand before it arrives.
6. When policies are combined, **the autoscaler always chooses the policy that results in the largest number of VMs to ensure availability**.

High Availability with MIGs: Autohealing and Regional Deployments

1. **Autohealing** automatically detects and recreates *unhealthy VMs* in a **Managed Instance Group**.
 - a. Autohealing **must be configured** with an **application-based health check to be effective**, as this checks if the **software** is working, not just the VM.
 - b. The **"initial delay"** setting is critical for autohealing to give new VMs time to boot before being health-checked.
2. A **Zonal MIG** runs all its VMs in a **single zone** and is vulnerable to a single point of failure if that zone fails.
3. A **Regional MIG** automatically distributes VMs across multiple zones in a region, providing high availability and fault tolerance.
4. **Regional MIGs are the best practice** for running reliable, production applications on Compute Engine.

Hands-On: Performing a Rolling Update on a MIG

1. We performed a **zero-downtime application update** using a **Managed Instance Group**.
2. The update process is triggered by **changing the instance template to the new version** and saving the changes to the MIG.
3. We used an **Automatic update** type to force the MIG to actively replace the old VMs immediately.
4. We configured **Maximum unavailable instances** to define how many old VMs could be offline at a time, which creates the "rolling" behavior.
5. We configured **Temporary additional capacity** (also known as "max surge") to control how many new VMs could be added above our target size, which is another way to manage the update process.
6. This entire process is **automated, safe, and ensures** *our application remains available to users throughout the deployment.*

A Practical Use Case: Building a Scalable Web Application

1. **Instance Template:** *The immutable "**blueprint**" for all of our web server VMs.*
2. **Regional MIG:** *The "**fleet manager**" that provides high availability by spreading VMs across multiple zones.*
3. **Autohealer:** *The "**mechanic**" that uses a conservative health check and an initial delay to replace individual failed VMs.*
4. **Autoscaler:** *The "**elastic controller**" that adds or removes VMs based on load (like CPU or traffic).*
5. **Application Load Balancer:** *The single "**front door**" that provides one static IP and distributes traffic **only** to the healthy VMs (as determined by its health check).*

Section 6: Google Kubernetes Engine (GKE): The Power of Containers

Containers vs. VMs: A Simple Analogy

1. VMs are heavier because they include a **full, dedicated Guest OS**, providing stronger isolation.
2. Containers are **lightweight** because they share the Host OS kernel, leading to better resource density.
3. Containers **start in seconds**, making them ideal for **rapid deployment** and **quick scaling up** and down.
4. **VMs are best for legacy applications** or when strong security mandates require complete OS-level separation.
5. The primary GCP service for running containers is Google Kubernetes Engine (GKE), which manages the container lifecycle.
6. **[Important]** In GCP, containers often run inside **Compute Engine Virtual Machines (the nodes)**, combining the isolation of the VM with the agility of the container.

The Core Concepts of Kubernetes: Pods, Services, Deployments

1. **A Pod** is the smallest deployable unit in **Kubernetes**.
 - a. It's a **wrapper** for one or more **containers** and provides a **shared network and storage**.
2. **A Deployment** is a "manager" that defines your desired state.
 - a. It tells Kubernetes how many replicas of a Pod you want to run.
 - b. Deployments provide **self-healing** by automatically replacing crashed Pods.
 - c. Deployments provide **rolling updates**, allowing you to update your application version with zero downtime.
3. **A Service** provides a stable, permanent IP address and DNS name for a group of **ephemeral Pods**.
 - a. Services act as an internal load balancer, distributing network traffic to all the healthy Pods that match its selector.

Introduction to GKE: Managed Kubernetes on GCP

1. **Google Kubernetes Engine (GKE)** is Google Cloud's managed Kubernetes service.
 - a. The primary benefit of GKE is that Google manages the cluster's **Control Plane** (the "brain") at minimal cost (with a free tier), which removes a huge operational burden.
 - b. You, the customer, are responsible for configuring and paying for the **Worker Nodes**, which are Compute Engine VMs that **GKE provisions to run your Pods**.
2. *GKE simplifies and automates cluster operations, including setup, scaling, security patching, and upgrades.*
3. Using GKE allows you to get all the power of **Kubernetes** (self-healing, scaling) without the complexity of managing the system itself.

GKE Modes: Autopilot vs. Standard Clusters

1. **GKE Standard** gives you full control over your worker nodes.
 - a. You must manage node pools and machine types.
2. **GKE Standard** is required for specialized hardware like GPUs (though Autopilot now supports them) or granular control over Spot VMs.
3. **GKE Standard's billing is per-node** (you pay for the whole VM), which can lead to over-provisioning and wasted cost.
4. **GKE Autopilot is a "serverless"** experience where Google manages everything, including the nodes.
5. **GKE Autopilot's billing is per-pod** (you pay only for the resources you request), **which is highly cost-efficient.**
6. **Choose:**
 - a. **Autopilot** for simplicity, reduced overhead, and most stateless applications.
 - b. **Standard** for maximum control and specialized hardware needs.

Hands-On: Deploying Your First GKE Autopilot Cluster

1. **GKE Autopilot mode** dramatically simplifies cluster creation.
2. Autopilot clusters are **'Regional' by default**, which gives you a highly available control plane at no extra configuration effort.
3. *In Autopilot, you do not create, manage, or configure any "Node Pools" or machine types.*
 - a. *This is all handled by Google.*
4. Because we don't manage nodes, our billing for Autopilot is based on the CPU and memory resources that our Pods request, not for the underlying VMs.
5. To connect to any GKE cluster, you use the **gcloud container clusters get-credentials...** command, which configures **kubectl** for you.

Hands-On: Deploying a Regional GKE Standard Cluster

1. **Standard mode** gives you full control over your nodes, including machine types, OS images, and scaling configurations.
2. In Standard mode, **you are responsible for creating and managing** one or more **Node Pools**.
3. A **Regional cluster** provides high availability by default, distributing your control plane and your nodes across three zones.
4. **Billing** for Standard mode is based on the underlying Compute Engine VMs, which run 24/7, whether you have Pods scheduled on them or not.
5. *Autopilot abstracts all node management away and bills per-Pod;*
6. *Standard exposes all node management to you and bills per-VM (Node).*

Security Matters: Understanding Private GKE Clusters

1. A **Private Cluster** isolates nodes and the control plane from the public internet.
2. You enable **Private Nodes** so nodes get only internal IPs.
3. Enabling private nodes **automatically creates a private control plane endpoint**, even if a public one remains.
4. The **Control Plane Access** section lets you enable the public and internal endpoints.
5. **Cloud NAT** handles outbound access to public sites.
6. **Private Google Access** handles communication with Google APIs (internally).
7. If a public endpoint remains, protect it using **Authorized Networks**.

The Kubernetes Command Line: Using kubectl

1. **kubectl** is the main command-line tool for interacting with a Kubernetes cluster.
 - a. It is pre-installed and ready to use in Google Cloud Shell.
2. You connect **kubectl** to a GKE cluster by running the ***gcloud container clusters get-credentials --location [LOCATION]*** command.
 - a. This command automatically updates your **kubeconfig file** and configures authentication.
3. **kubectl get [resource]** is used to list resources.
 - a. You must know:
 - i. *kubectl get nodes*
 - ii. *kubectl get pods*
 - iii. *and kubectl get services*
4. **kubectl describe [resource] [name]** is the most important command for troubleshooting and getting detailed information about a specific resource.

Getting Your App Ready: Containerizing an Application with Docker

1. A **container** solves the *"it works on my machine"* problem by bundling an application with all its dependencies.
2. A **Dockerfile** is the text file recipe used to define the steps to build a container image.
3. An **Image** is the read-only, shareable template you build from a Dockerfile.
4. A **Container** is a live, running instance of an image.
5. The **docker build** command is used to create an image from a Dockerfile.
6. You must be able to recognize key Dockerfile instructions:
 - a. *FROM (base image)*
 - b. *WORKDIR (set directory)*
 - c. *COPY (add files)*
 - d. *RUN (install dependencies)*
 - e. *CMD (run the app)*

Storing Your Containers: Introduction to Artifact Registry

1. **Artifact Registry** is Google's modern, recommended service for storing container images and other software packages.
 - a. *It has replaced the older, deprecated **Container Registry (gcr.io)**, which was fully shut down in 2025.*
 - b. *It is a multi-format registry, supporting Docker, Maven, npm, Python, and more.*
2. **Repositories are regional** (e.g., us-central1), which improves performance and data control.
3. The standard naming format for Docker is **[LOCATION]-docker.pkg.dev**
4. You control access using standard **IAM roles** like *Artifact Registry Reader* (to pull) and *Writer* (to push).

Hands-On: Pushing a Container Image to Artifact Registry

1. *First*, we enabled the **artifactregistry.googleapis.com** API.
2. *Second*, we created our repository with “**gcloud artifacts repositories create**”, specifying the format as **docker** and a **location**.
3. *Third*, we authenticated Docker using “**gcloud auth configure-docker**”.
4. *Fourth*, we tagged our local image with its full repository path.
5. *Fifth*, we used “**docker push**” to upload our image.

Hands-On: Deploying Your First Application to GKE

1. **kubectl create deployment [NAME] --image=[IMAGE-PATH]** is the imperative command to deploy an application.
2. A **Deployment** is the Kubernetes resource that manages your application and its desired state
 - a. Example: *"I want 3 copies of this image running"*
3. A **Pod** is the object that actually runs your container. **Deployments create and manage Pods.**
4. **You must use the full image path** from **Artifact Registry** (e.g., location-docker.pkg.dev/...) in your Deployment.
5. The GKE node service account needs **Artifact Registry Reader role** to pull images.
6. By default, **Pods are only accessible inside the cluster** and are not exposed to the internet.
7. To expose a **Deployment**, you must create a **Service**.

Hands-On: Exposing Your Application with Kubernetes Services

1. A **Service** provides a stable network endpoint (a "front desk") for a set of changing Pods.
 - a. **type=ClusterIP** is for internal-only communication.
 - b. **type=LoadBalancer** exposes your app to the internet by provisioning an external passthrough Network Load Balancer.
2. "**kubectl expose deployment**" is the imperative command to create a Service for an existing Deployment.
 - a. **--port** is the port the Service listens on (e.g., 80 for web).
 - b. **--target-port** is the port your container is listening on (e.g., 8080 from our Dockerfile).

Section 7: Managing and Scaling GKE Clusters

Viewing Your Cluster: Exploring Nodes, Pods, and Services

1. **kubectl get nodes** shows you the health and status of your worker machines, the "factory floor."
2. **kubectl get pods** lists your running applications and their ready status (1/1) and overall STATUS (like Running).
3. **kubectl get services (or svc)** shows your application's network endpoints, including the **EXTERNAL-IP** for **LoadBalancer** types.
4. **kubectl describe [resource] [name]** is your most powerful inspection tool for troubleshooting and seeing detailed information.
5. When debugging a Pod, always check the **Events** section of **kubectl describe pod** for error messages.
6. When debugging a Service, always check the **Endpoints** field of **kubectl describe service** to see if the Service is finding your Pods.

Understanding GKE Node Pools

1. A **Node Pool** is a group of nodes in a **GKE Standard** cluster that all share the same configuration (machine type, disk, etc.).
2. Node pools are the primary way to support **heterogeneous workloads** (mixed applications) within a single cluster.
3. You can create separate node pools for specialized hardware like **GPUs** or for cost-savings using **Spot VMs**.
4. [Reminder] In **GKE Autopilot**, you do not create or manage node pools; this is all handled by Google.
5. The ACE exam specifically expects you to know how to *"add, edit, or remove a node pool."*

Hands-On: Adding, Editing, and Removing a Node Pool

1. You manage node pools for GKE Standard clusters from the **"Nodes" tab**.
2. **We added** a new node pool by clicking **"Create a user-managed node pool"** and specifying a different machine type, proving we can support heterogeneous workloads.
3. **We edited** the node pool by clicking **"Resize"** and changing the node count per zone, which is a form of manual scaling.
4. **We removed** the node pool by clicking **"Delete,"** which is the proper way to decommission unused infrastructure and save costs.
5. **You can delete any node pool in GKE**, including the default one, **as long as the cluster has at least one remaining node pool**.
 - a. The last node pool in a cluster **cannot** be deleted.

Scaling Your Workloads Horizontal (HPA) and Vertical (VPA) Pod Autoscaling - 1

1. **HPA (Horizontal)** scales **out**. *It adds more Pods to handle traffic.*
 - a. Think: "opening more cashier lanes."
 - b. HPA is triggered by a metric, most commonly CPU Utilization, and is perfect for reacting to spiky traffic.
2. **VPA (Vertical)** scales **up**. *It changes a single Pod's CPU or memory request.*
 - a. Think: "giving a cashier a faster scanner."
 - b. VPA's primary goal is "right-sizing" applications to make them more efficient and prevent "Out Of Memory" errors.
 - c. VPA requires a Pod restart to apply its changes, making it disruptive.

Scaling Your Workloads Horizontal (HPA) and Vertical (VPA) Pod Autoscaling - 2

6. **You cannot use** HPA (on CPU/memory) and VPA (in "Auto" mode) on the same Deployment.
 - a. They will conflict.
7. You can use VPA in "Off" (recommendation-only) mode to find the right size, and then apply HPA to that well-sized pod.
8. **Exam Note:** **VPA is enabled by default in Autopilot clusters**, but must be **manually enabled** in Standard clusters.

Scaling Your Infrastructure: Cluster and Node Pool Autoscaling

1. **The Cluster Autoscaler (CA)** is the "Layer 2" infrastructure scaling solution.
 - a. *It adds and removes Nodes (VMs) from your node pools.*
 - b. The CA works with the HPA (Layer 1, Pod scaling) to create a **fully elastic system**.
 - c. The CA is not triggered by Node CPU/Memory.
 - i. It is triggered only by **Pending Pods** that cannot be scheduled.
 - d. **The CA scales down by identifying underutilized nodes, draining them, and terminating them to save money.**
2. In **GKE Standard**, you must enable and configure the **Cluster Autoscaler** on a *per-node-pool* basis, setting a **min** and **max** size.
3. In **GKE Autopilot**, the **Cluster Autoscaler does not exist for you**; *node scaling is fully managed by Google.*

Hands-On: Configuring HPA for a Deployment

1. We configure the Horizontal Pod Autoscaler (HPA) from the Workloads page of our Deployment.
 - a. **HPA requires a metric** (like CPU utilization), a target value (like 60%), and min/max replicas to act as guardrails.
2. **An Important Requirement:** our deployment must have **resource requests defined**.
 - a. Without them, the HPA cannot calculate utilization percentages and will show *"Unable to read all metrics."*
3. We fixed this by **patching our deployment** with CPU and memory resource requests using **kubectl**.
4. We can verify it works in the console or by running **kubectl get hpa** and **kubectl top pods**.
5. For a complete solution, we must also enable the **Cluster Autoscaler** on our Node Pool, ensuring our infrastructure can grow along with our application.

Managing Stateful Applications with Kubernetes StatefulSets

1. **Deployments** are for stateless applications where pods are identical and replaceable.
 - a. Use these for web **servers**, **APIs**, and **worker processes**.
2. **StatefulSets** are for stateful applications that need stable identities and persistent storage.
 - a. Use these for **databases**, **message queues**, and applications that store data.
 - b. **StatefulSets provide:**
 - i. Stable network identity
 - ii. Stable persistent storage
 - iii. Ordered deployment
3. **GKE automatically provisions persistent disks** *through dynamic provisioning when you create PVCs.*
4. Deleting a StatefulSet **does NOT delete** its persistent volumes - this protects your data.

Securely Accessing GCP Services from GKE Pods

1. **Workload Identity Federation for GKE** (often just called **Workload Identity**) *is the recommended, secure way for GKE Pods to access GCP services.*
 - a. It replaces insecure methods like using static **JSON keys** or **overly broad node permissions**.
 - b. It works by creating a one-to-one mapping (**binding**) between a **Kubernetes Service Account (KSA)** and a **Google Service Account (GSA)**.
2. *The token exchange happens automatically via the GKE metadata server and works seamlessly with Google Cloud client libraries, requiring no code changes for most apps.*
3. It is **enabled by default** on **Autopilot clusters** but **must be manually enabled** on **Standard clusters** (at least for now 😊).

Section 8: Serverless Computing with Cloud Run (Services & Functions)

What is Cloud Run? From Container to Scalable Service in Seconds

1. **Cloud Run** is a **fully managed serverless platform** that runs stateless containers.
 - a. its most famous feature is the ability to scale down to zero, making it **extremely cost-effective** for variable or infrequent workloads.
2. You are billed only for the resources (CPU and memory) used **during request processing**.
3. **Cloud Run** simplifies deployment.
 - a. The input is a **Container Image**, and the output is a **scalable HTTPS service**.
4. **Remember:** Cloud Run abstracts all infrastructure management. You do not manage servers, node pools, or Kubernetes clusters.

Hands-On: Deploying a Container to Cloud Run

1. To deploy a container to **Cloud Run**, you start by navigating to the **Cloud Run page** and clicking "**Deploy container**" (or "Create Service").
 - a. You must select a container image from a registry, like **Artifact Registry** or **Docker Hub**.
 - b. You must assign a **Service name** and a **Region** for your service.
 - c. For **Authentication**, you choose between *"Allow public access"* (unauthenticated) or *"Require authentication"* (private, IAM-controlled).
 - d. For **Service scaling**, a minimum of 0 instances enables the scale-to-zero feature. Setting a maximum is a key cost-control guardrail.
 - e. For **Ingress**, "All" allows public internet traffic, while "Internal" restricts access to your VPC.
 - f. You must configure the **Container port** to match the port your application listens on, for example, 8080.

Managing Traffic: Revisions and Traffic Splitting in Cloud Run

1. To deploy a new revision without sending traffic to it, you click "**Edit & Deploy New Revision**", select your new image, and **unchecked** "***Serve this revision immediately***".
2. This deploys the new revision in a "healthy" state but with **0% traffic**.
3. To set up a traffic split, you go to the "**Revisions**" **tab** of your service and click "**Manage traffic**".
4. This allows you to assign specific percentages to different revisions, which is how you perform a **canary release**.
5. To perform an **instant rollback**, you simply go back to "Manage traffic" and set your stable, old revision back to **100%**.

Configuring Autoscaling for Cloud Run Applications

1. **Cloud Run autoscaling** works by adding or removing **container instances** based on incoming traffic.
2. The **Minimum Number of Instances** setting controls cold starts.
 - a. *Setting it to 0 (the default) enables scale-to-zero, which is cheapest.*
 - b. *Setting it to 1 keeps an instance "warm" to reduce latency.*
3. The **Maximum Number of Instances** is your most important cost-control guardrail to prevent runaway scaling and unexpected bills.
4. **Concurrency** is the number of requests a single instance can handle at the same time.
 - a. *You use high concurrency (like 80) for I/O-bound applications (like most web APIs) to save costs.*
 - b. *You use low concurrency (like 1) for CPU-bound applications (like image processing) to ensure performance.*

What are Cloud Run functions? Event-Driven Compute

1. **Cloud Run functions** are small, single-purpose pieces of code designed for event-driven compute.
 - a. This is different from **Cloud Run services**, *which are containerized applications designed for the request-response (HTTP) model.*
2. Functions are **triggered** by **events** from other GCP services, *such as a file upload to Cloud Storage or a message to a Pub/Sub topic.*
3. With **functions**, **you deploy your source code**, and Google Cloud **automatically builds the container for you using Buildpacks**.
4. With **services**, *you must build and provide the container image yourself.*
5. Functions are often used as "**glue**" to connect different GCP services and automate workflows.
6. When/if you see an exam question about *"running code in response to a new file in a bucket,"* your brain should immediately think: **"Cloud Run function."**

Cloud Run functions Use Cases: Responding to Events

1. **Key Trigger 1: Cloud Storage:** *When you see a question about processing files after they are uploaded, the answer is Cloud Run functions triggered by Cloud Storage.*
 - a. This is perfect for resizing images, analyzing data, or generating reports.
2. **Key Trigger 2: Pub/Sub:** *When you see a question about "decoupling services" or "asynchronous processing," the answer is Cloud Run functions triggered by a Pub/Sub topic (using Eventarc).*
 - a. This is the "glue" for resilient microservices.
3. **Key Trigger 3: Cloud Logging (via Pub/Sub):** For any question about "real-time alerts" or "automating responses to logs" (like security or error logs), the answer is Cloud Run functions.
 - a. The flow is: Log Sink -> Pub/Sub -> Eventarc Trigger -> Cloud Run functions.
4. **The Core Concept:** In every case, Cloud Run functions act as the serverless "glue" that connects a service's event (the trigger) to a custom piece of code (the action).
5. **The Goal:** These patterns allow you to build complex, automated systems that only run when needed, which is the very definition of cost-efficient, event-driven computing.

Hands-On: Creating a Cloud Run Function Triggered by a Cloud Storage Upload

1. To create an event-driven function, we use the Cloud Run **"Create service" workflow** with the **"Function"** option.
 - a. This creates a Cloud Run service, but with a simplified UI for functions.
2. **We created a dedicated service account** *with only the specific permissions needed (Eventarc Event Receiver, Storage Object Viewer, Cloud Run Invoker, and Storage Object Creator on the output bucket) - this follows the principle of least privilege.*
3. The **Trigger** is configured during creation using the "Add trigger" button, which configures Eventarc behind the scenes.

Hands-On: Creating a Cloud Run Function Triggered by a Cloud Storage Upload

5. We configured service accounts in **TWO** places: the trigger's service account (in Eventarc config) and the function's runtime service account (in Security tab) - both use our custom *cloud-function-processor service account*.
6. We **matched our bucket and trigger regions (us-central1)** - this is required for **Eventarc** to work.
 - a. The event type was ***google.cloud.storage.object.v1.finalized***
7. We deployed source code (Python) and requirements.txt, and Google Cloud used **Cloud Build** to *automatically containerize it*.
8. Our new function appears in the main **Cloud Run** services list because functions are just specialized Cloud Run services.
9. This **"two-bucket" pattern** (one for input, one for output) is a best practice for all data processing pipelines.

Connecting Events Across GCP: An Introduction to Eventarc

1. **Eventarc** is a fully managed service that routes events from a source (provider) to a destination (receiver).
 - a. For the ACE exam, the **primary event receivers** you should know are Cloud Run and Cloud Run functions.
 - b. Eventarc supports **two types of triggers**:
 - i. Fast **"Direct"** events from sources like **Cloud Storage** and **Pub/Sub**.
 - ii. **"Cloud Audit Log"** events, which let you create triggers from over **130 Google Cloud providers** that do not have direct events.
2. The "trigger" we created in our last demo was a direct Eventarc trigger for a **Cloud Storage event**.
3. *Using Cloud Audit Logs allows you to build automation for security, operations, or any other task in response to actions in services like BigQuery, Compute Engine, or IAM.*

Compute Face-Off: VMs vs. GKE vs. Cloud Run

1. **Compute Engine** - Maximum Control

- a. *Use it for "lift-and-shift" legacy applications*
- b. *Use when you must manage the operating system directly*

2. **GKE** - Container Orchestration

- a. *Use it for complex, stateful, or multi-container applications*
- b. *Use when you need the full power and portability of the Kubernetes API*

3. **GKE Autopilot** - Kubernetes Without the Complexity

- a. *Removes the need to manage nodes*
- b. *Now offers a container-optimized platform, GPU support*
- c. *Blends the power of Kubernetes with serverless-like simplicity*

Compute Face-Off: VMs vs. GKE vs. Cloud Run

4. **Cloud Run Services** - Stateless, Serverless Containers

- a. Use them for HTTP web applications and APIs*
- b. Perfect when you need to scale to zero*

5. **Cloud Run functions** - Event-Driven, Serverless Code

- a. Now part of the Cloud Run platform*
- b. Use them as "glue" to react to internal GCP events*
- c. Example: responding to a Cloud Storage upload*

Section 9: Cloud Storage: Your Infinitely Scalable Object Store

What is Object Storage? A Storage Shelf Analogy

1. Object Storage (**Google Cloud Storage - GCS**) stores data as **objects** inside **buckets**.
 - a. An object consists of the **file data itself** plus **metadata** (info about the file—like a label on a box).
 - b. Standard buckets use a flat namespace.
 - i. Any "folders" you see in the console are simulated using the object's name (prefix).
2. Objects are uniquely identified by **their bucket, their name** (key), and their **generation number**.
3. GCS provides **11 9s of durability** (99.999999999%), meaning the **risk** of losing data is *virtually zero*.
4. **GCS is the preferred storage** for unstructured data like *images, videos, backups, and logs*.

Hands-On: Creating Buckets and Uploading Objects

1. **Bucket Names:** Must be **globally unique**.
2. **Uniform Bucket-Level Access:** This is the standard.
 - a. **Important Exam Tip:** Once you enable Uniform access, you have **90 days** to revert it.
 - i. After 90 days, it becomes permanent and you cannot switch back to Fine-grained.
3. **Soft Delete:** New buckets now have Soft Delete enabled by default with a **7-day retention period**.
 - a. *This allows you to restore deleted objects.*
4. **Public Access Prevention:** This setting overrides IAM. Even if you grant a user "Storage Object Viewer" permissions, if Public Access Prevention is on, the public cannot see the files.
5. **Immutability:** You cannot "edit" an object in place.
 - a. *You must upload a new version to overwrite it.*

Hands-On: Managing Objects with gsutil and gcloud storage CLI

1. **gsutil: The legacy tool.**
 - a. *It still works, but it's slower and lacks new features.*
 - b. *Recognizable by short commands like **mb**, and flags like **-r**, **-d**.*
2. **gcloud storage: The modern, recommended tool.**
 - a. It is **faster** and uses **explicit command structures**.
3. **buckets create:** Replaces **mb**. (*gsutil **mb** → gcloud storage buckets create*)
4. **cp:** Copy. Used for uploads and downloads.
5. **rsync:** Synchronize.
 - a. **--recursive:** *Copies directory trees (replaces **-r**).*
 - b. **--delete-unmatched-destination-objects:** *Makes the destination exactly match the source, deleting extra files (replaces **-d**).*
6. **Performance: gcloud storage is significantly faster** (up to 94% faster on downloads) due to automatic parallelization.

Cloud Storage Classes: Finding the Right Price and Performance

1. **Standard:** Best for "hot" data. No minimum duration. No retrieval fee.
2. **Nearline:** Best for data accessed once a month. 30-day minimum.
3. **Coldline:** Best for data accessed once a quarter. 90-day minimum.
4. **Archive:** Best for long-term compliance accessed less than once a year. 365-day minimum.
5. **Early Deletion Fee:** If you delete data from colder classes before the minimum duration is up, you pay for the remaining time.
6. **Autoclass:** Automates transitions.
 - a. **Remember:** Small files (<128 KiB) are ignored, and by default, it only transitions to Nearline (unless you configure it for Archive).

Securing Your Buckets: Permissions and Signed URLs

1. **IAM** is for **internal users** and **service accounts**.
 - a. *Principle of Least Privilege*: Grant Storage Object Viewer (read-only) instead of Storage Admin whenever possible.
2. **Uniform Bucket-Level Access** is the standard.
 - a. **It disables legacy ACLs** and is required for features like **Managed Folders**.
3. **Signed URLs** are for **external users** (like customers).
 - a. **Time-Limited**: Signed URLs are **valid for a specific duration** (e.g., 15 minutes). They are not strictly "one-time use," but they **expire automatically**.
 - b. **Direct Access**: Signed URLs allow users to upload/download directly to Cloud Storage, bypassing your web server.

Object Lifecycle Management - Automating Data Transitions

1. **Object Lifecycle Management** automates transitioning storage classes or deleting objects based on rules (If This, Then That).
 - a. **Conditions:** *You can filter by Age, CreatedBefore, IsLive (for versioning), and MatchesPrefix/Suffix (for file types or folders).*
2. **Cost Advantage:** Using the SetStorageClass action typically avoids early deletion fees that you would incur if you moved the data manually.
3. **Minimum Durations:** Even with automation, remember the targets:
 - a. *Nearline: 30 days.*
 - b. *Coldline: 90 days.*
 - c. *Archive: 365 days (Do not delete from Archive before a year).*
4. **Soft Delete:** A lifecycle "Delete" action respects your Soft Delete policy.
 - a. The data enters the soft-delete state for the configured duration (default 7 days) before permanently disappearing.

Hands-On: Configuring Lifecycle Policies

1. **Console Tab:** Lifecycle policies are managed in the Lifecycle tab of the bucket details.
2. **Action vs. Condition:** Know the difference.
 - a. **Action** is what happens (Delete, SetStorageClass).
 - b. **Condition** is when it happens (Age, matchesPrefix, etc).
3. **JSON Syntax:** Remember that JSON keys use camelCase (e.g., age, matchesPrefix, matchesStorageClass).
4. **The Command:** To apply a policy, use:
 - a. ***gcloud storage buckets update*** with the ***--lifecycle-file*** flag.
5. **Eventually Consistent:** Just because the command finished, doesn't mean the changes are instant. It can take up to 24 hours for the new configuration to propagate.
 - a. During this time, Google Cloud might still perform actions based on your old configuration.

Object Versioning and Retention Policies

1. **Object Versioning** provides an "undo" button by creating "non-current" (historical) versions when you delete or overwrite a file.
 - a. The "live" version is the current object; "non-current" versions are the old ones.
2. **Cost Warning:** *You pay for storage for ALL versions.*
 - a. To manage costs, use **Lifecycle Management** with conditions like **isLive: false** or **numNewerVersions** to automatically delete old versions.
3. **Retention Policies (Bucket Lock)** are for legal or compliance needs.
 - a. They make data immutable (undeletable) for a fixed period.
 - b. A locked Retention Policy cannot be removed or shortened by anyone, but the duration can be increased.
4. **Versioning = Recovery.**
5. **Retention = Compliance.**

What is Storage Transfer Service?

1. **When to use Storage Transfer Service (STS):** *Large transfers into GCS (typically > 1 TiB).*
 - a. **Cloud-to-Cloud:** *STS can pull directly from AWS S3, Azure Blob, or S3-compatible systems.*
 - b. **On-Prem:** *Install transfer agents inside your network and group them in an agent pool.*
2. **Scheduling:** *Run once or on a recurring schedule.*
3. **Synchronization:** *Incremental by default—only new or changed files.*
4. **Data Integrity:** *Automatic end-to-end checksum validation.*

Estimating Cloud Storage Costs with the Pricing Calculator

1. **Tool of Choice:** To estimate costs, you must use the **Google Cloud Pricing Calculator**.
 - a. **Cost 1: Data Storage:** *The cost per GB for data "at-rest," determined by **Storage Class** and **Location**. (Includes replication fees for multi/dual-regions).*
 - b. **Cost 2: Network Usage:** *Also called **Internet Data Transfer** (or Egress). Ingress is free.*
 - i. *Be careful with **Region-to-Multi-Region** transfers!*
 - c. **Cost 3: Operations:** *Class A (writes/lists) and Class B (reads).*
 - i. ***Hierarchical** namespaces (folders) cost more.*
 - d. **Cost 4: Retrieval Fees:** *A separate, per-GB fee for reading data from Nearline, Coldline, or Archive classes.*

Section 10: Managed Databases in GCP

The GCP Database Landscape: Relational vs. NoSQL

1. **GCP Database Landscape:** *A suite of fully managed services where Google handles the operational maintenance (OS, patching, hardware), allowing you to focus on data and users.*
 - a. **Relational (SQL) Options:** *Best for **structured, tabular data**. Use **Cloud SQL** for general workloads, **AlloyDB** for high-performance **PostgreSQL**, or Spanner for global scale.*
2. **NoSQL Options:** *Best for **unstructured** or **semi-structured data** requiring high flexibility and scalability.*
 - a. **Firestore:** *A serverless NoSQL document database, perfect for mobile and web application backends.*
 - b. **Bigtable:** *A high-throughput NoSQL database designed for massive analytics and IoT workloads (Petabyte scale).*
 - c. **Memorystore:** *An in-memory data store (Redis/Memcached) used primarily for caching to improve application performance.*

Hands-On: Creating a Cloud SQL Instance

1. We configured and deployed a **Cloud SQL instance**.
2. **Multiple Zones** configuration creates a standby replica in another zone and enables automatic failover, which is required for production high availability.
3. **Enable automatic storage increases** is the key setting to prevent database crashes from full disks, as it allows Google to expand storage capacity automatically.
4. **Private IP** keeps databases secure within your VPC, while **Public IP** requires **authorized network ranges** to control which IP addresses can connect.
5. **Point-in-time recovery** uses binary logs to restore your database to any specific moment, not just to the last daily backup, and is automatically enabled with **high availability**.
6. **Backup windows** should be scheduled during low-traffic periods, and the **retention period** determines how many days of **backups** and **binary logs** are kept for recovery.

Cloud SQL: Your Managed MySQL, PostgreSQL, and SQL Server

1. **Cloud SQL Engines:** A fully managed relational database that supports:
 - a. **MySQL**
 - b. **PostgreSQL**
 - c. **SQL Server**
2. **Managed Service:** Automates backups, patches, and OS updates.
3. **Scaling:** Scales **Vertically** (bigger machine) for writes. It can scale reads using Read Replicas.
4. **Storage:** Supports Automatic Storage Increases so you don't run out of disk space.
5. **Regional:** It lives in a specific region (e.g., us-east1). **It is NOT global by default.**
6. **Use Case:** Best for general web frameworks (WordPress, Django), ERPs, CRM, and lifting-and-shifting existing relational databases.

Hands-On: Creating a Cloud SQL Instance

1. In this **demo**, we...

- a. **Provisioned a Managed Database:** *We deployed a fully managed PostgreSQL instance using the Cloud SQL Enterprise Edition.*
- b. **Ensured Scalability:** *We enabled Automatic Storage Increase. This is critical for the exam because it prevents your database from crashing if the disk gets full.*
- c. **Configured Networking:** *We assigned a Public IP and authorized our network to allow connections. Remember, for a real exam or production scenario, Private IP is the more secure standard.*
- d. **Verified Connectivity:** *Finally, we used the `gcloud` command within Cloud Shell to log in and confirm our database is ready for action.*

Hands-On: Creating a Cloud SQL Instance

1. In this **demo**, we...

- a. **Provisioned a Managed Database:** *We deployed a fully managed PostgreSQL instance using the Cloud SQL Enterprise Edition.*
- b. **Ensured Scalability:** *We enabled Automatic Storage Increase. This is critical for the exam because it prevents your database from crashing if the disk gets full.*
- c. **Configured Networking:** *We assigned a Public IP and authorized our network to allow connections. Remember, for a real exam or production scenario, Private IP is the more secure standard.*
- d. **Verified Connectivity:** *Finally, we used the `gcloud` command within Cloud Shell to log in and confirm our database is ready for action.*

High Availability and Read Replicas in Cloud SQL

1. **Cloud SQL Architecture:** You have two main architecture choices:
 - a. High Availability (HA) for reliability and
 - b. Read Replicas for performance.
2. **High Availability (HA):** Creates a Standby instance in a different Zone for automated failover during an outage.
3. **Standby Limitation:** You cannot read or write to the Standby instance; it sits **idle** until a failover occurs.
4. **Read Replicas:** Active copies of the database used to scale **Read traffic** and reduce latency for users.
5. **Cross-Region:** Read Replicas can be placed in different regions (e.g., Primary in US, Replica in Europe) to bring data closer to users.
6. **Promotion:** Replicas do not failover automatically; they must be **manually** Promoted to become a standalone **Primary instance**.

Backup, Restore, and Multi-Region Redundancy in Cloud SQL

1. **Cloud SQL Backups:** Automated backups run daily and are incremental, while On-Demand backups are *manual* and persist *until you delete* them.
2. **Point-in-Time Recovery (PITR):** Uses *transaction logs* to restore your database to a specific millisecond in the past.
3. **Restoration Rule:** Restoring a Cloud SQL instance ALWAYS creates a NEW instance.
 - a. It never overwrites the existing one.
4. **Disaster Recovery (DR):** High Availability handles Zone failures; Cross-Region Read Replicas are required to survive Region failures.
5. **Promotion:** To failover to a different **region** during a disaster, you must **manually Promote the Read Replica** to be the new Primary.

Spanner: The Globally Consistent, Infinitely Scalable Relational Database

1. **Cloud Spanner** is a *fully managed, globally distributed relational database with strong consistency*.
2. **Scaling: Horizontal** scaling using Nodes (**1 node = 1,000 Processing Units**).
 - a. *Each node supports up to 10 TB of storage.*
3. **TrueTime**: *Uses atomic clocks and GPS to guarantee global transaction ordering—this enables External Consistency.*
4. **Availability**: *99.99% SLA for regional instances, 99.999% SLA for multi-region instances.*
5. **Use Cases**: *Global financial systems, multi-region gaming, supply chain management—anything requiring ACID compliance at global scale.*
6. **Dialects**: *Google Standard SQL and PostgreSQL.*

Firestore: A Flexible, Scalable NoSQL Document Database

1. **Firestore** is Google's **fully managed, serverless, NoSQL document database**.
2. **Data Model:** Uses **Collections** and **Documents** (JSON-like). Flexible schema (Schemaless).
3. **Modes:** You must choose between...
 - a. **Native Mode** (Mobile/Web, Real-time, Offline)
 - b. **Datastore Mode** (Server-side, Legacy support)
4. **Serverless:** Scales to zero. No instances to manage.
5. **Features:** Built-in Offline Persistence and Real-time Sync for mobile/web clients.
6. **Pricing:** Pay-per-operation (Read, Write, Delete) and storage. Not paid by "node hour."
7. **Anti-Pattern:** Do not use Firestore for high-frequency updates to a single document (more than 1 per second). Use Bigtable or Memorystore for that.

Bigtable: The Petabyte-Scale NoSQL Database for Analytics

1. **Cloud Bigtable** is Google's fully managed, wide-column NoSQL database designed for massive scale.
2. **Use Case:** Massive scale (Petabytes), high throughput (millions of writes/sec), and flat data (IoT, AdTech, Financial history).
3. **Anti-Pattern:** Do NOT use Bigtable for less than 1TB of data or structured transaction data (use Cloud SQL).
4. **Architecture:** Separates Compute (Nodes) from Storage (Colossus).
 - a. This allows instant scaling without data migration.
5. **Performance:** Scales linearly.
 - a. Adding nodes increases performance proportionally.
6. **Row Key Design:** The most critical factor.
 - a. Avoid "Hotspots" by avoiding sequential keys (like timestamps) at the start of the Row Key.
7. **API:** Compatible with Apache HBase.

AlloyDB: A High-Performance, PostgreSQL-Compatible Database

1. **AlloyDB** is a *fully managed, PostgreSQL-compatible database service* for demanding enterprise workloads.
2. **Performance:** *It is 4x faster for transactions and 100x faster for analytics compared to standard PostgreSQL.*
3. **Architecture:** Uses **Disaggregated Compute** and **Storage**.
 - a. *Compute nodes are stateless and can scale independently of storage.*
4. **Columnar Engine:** An in-memory accelerator that speeds up analytical queries.
 - a. *Enables HTAP (Hybrid Transactional/Analytical Processing).*
5. **Use Case:** Choose AlloyDB when you need **high-throughput PostgreSQL performance** or need to run analytics on **real-time data**.
6. **Compatibility:** It is 100% compatible with open-source PostgreSQL tools and libraries.

In-Memory Speed: An Introduction to Memystore

1. **Memystore** is a fully managed in-memory service for **Redis**, **Memcached**, and **Valkey**.
 - a. Provides sub-millisecond latency.
2. **Use Case:** Caching to reduce database load, Session management, Gaming leaderboards, Real-time data access.
3. **Engines:**
 - a. **Redis:** Complex data structures, Persistence (RDB/AOF), High Availability.
 - b. **Memcached:** Simple key-value, Multi-threaded, No persistence.
 - c. **Valkey:** Open-source, Cross-region replication, Up to 250 shards, 99.99% SLA.
4. **Tiers (Redis):**
 - a. **Basic:** Single node, data preserved during maintenance, cheaper.
 - b. **Standard:** High Availability with automatic failover, 99.9% SLA.

In-Memory Speed: An Introduction to Memorystore

5. **Scaling:** Redis standalone scales up to 300 GB.
 - a. Redis Cluster and Valkey **scale horizontally** to terabytes with zero downtime.
6. **Networking:** Private IP only (accessible only within the VPC).

Hands-On: Executing Basic Queries Against a Cloud SQL Database

1. **Cloud SQL Studio vs. CLI:**

- a. **Cloud Shell (gcloud sql connect):** Quick connectivity checks and scripting with automatic IP allowlisting
- b. **Cloud SQL Studio:** Web-based SQL editor requiring IAM roles (Cloud SQL Studio User or Admin) plus database credentials

2. **Connectivity in Production:** *Production applications don't use Cloud SQL Studio or gcloud commands. Instead, they connect via **Cloud SQL Auth Proxy** or **private VPC IPs**—we'll cover these in the networking section.*

3. **Data Persistence:** Your data persists through instance stops and starts. But if you delete the instance, the data is gone forever—unless you have backups!

Backup and Restore Operations Across GCP Databases

1. Need **automated backups** with **PITR (Point-in-Time Recovery)**? → **Cloud SQL** or **AlloyDB**
2. Need **global consistency with fast recovery**? → **Spanner**
3. Working with **NoSQL documents**? → **Firestore** (but plan manual exports)
4. Handling **petabyte-scale analytics**? → **Bigtable** (but manual exports required)

Centralized View - Using the Database Center

1. **Centralized Monitoring:** *You can view CPU, memory, storage, and connection metrics for all databases without navigating to individual service consoles.*
2. **Health and Compliance:** *Database Center shows security recommendations, backup status, and compliance posture across your entire database portfolio.*
3. **Quick Access:** *Click on any database in the list to jump directly to its detailed configuration page in the respective service console.*
4. **Cost Visibility:** *See estimated monthly costs for each database instance, helping you identify optimization opportunities.*

Estimating Database Costs - Critical Cost Factors

1. *Over-provisioned instances?* **Right-size them.**
2. *Running HA when not needed?* **Switch to single-zone.**
3. *High read traffic?* **Add read replicas instead of scaling up the primary.**
4. *Firestore with massive read operations?* **Consider caching with Memorystore.**

Section 11: Big Data & Analytics Platforms

BigQuery: GCP's Serverless Data Warehouse

1. **BigQuery** is a fully managed, AI-ready data platform that helps you manage and analyze your data.
2. **Service Type:** Serverless, multi-cloud data warehouse (OLAP).
3. **Architecture:** *Separates storage (Colossus/Capacitor) from compute (Slots).*
4. **Data Loading:** You can load data via the **Cloud Console**, from local files using **bq load**, via **Streaming inserts** for real-time data, or using the **BigQuery Data Transfer Service** for automated transfers.
5. **Best Practice:** Avoid **SELECT *** to save money on query processing costs.

Hands-On: Loading Data into BigQuery

1. **Creating Datasets:** Remember that Dataset location is permanent.
 - a. You cannot change a dataset from "US" to "EU" after creation.
2. **bq load:** Memorize the bq load command.
 - a. Remember the **--autodetect flag** is your friend for quick loads.
3. **Formats:** BigQuery supports **CSV**, **JSON** (newline delimited), **Avro**, **Parquet**, and **ORC**.
 - a. **Avro** is often the fastest for loading.
4. **Transfer Services:**
 - a. **Storage Transfer Service** = Source -> GCS Bucket.
 - b. **BigQuery Data Transfer Service** = Source/GCS -> BigQuery Table (Automated/Scheduled).

Reviewing Job Status and Query Performance in BigQuery

1. **Troubleshooting:** Use **Job History** in the console or **bq show** in the CLI to see why a specific job failed.
2. **Audit & Metrics:** Query **INFORMATION_SCHEMA.JOBS** to programmatically analyze costs, user activity, and error rates across your project.
3. **Performance:**
 - a. **Partitioning** = Splits a single table into segments by Date/Time (reduces cost).
 - b. **Clustering** = Sorts data inside partitions (improves specific lookups).
 - c. **Anti-Pattern:** *Never use `SELECT *` in production.*

Pub/Sub: The Global Messaging Service for Event-Driven Systems

1. **Decoupling:** *Use Pub/Sub to buffer data between systems that process at different speeds (e.g., IoT sensors sending data to a slow database).*
2. **Push vs. Pull:**
 - a. **Pull:** Best for large batch processing (Dataflow).
 - b. **Push:** Best for real-time webhooks (Cloud Run/Cloud Run functions).
3. **Standard vs. Lite:**
 - a. **Standard:** Global, auto-scaling, higher cost.
 - b. **Lite:** Zonal, manual capacity planning, lower cost **(Deprecated)**.
4. **Exactly-Once Delivery:** Pub/Sub supports "exactly-once" delivery for pull subscriptions only, ensuring you don't process the same duplicate message twice within a cloud region.

Hands-On: Pub/Sub with Cloud Run Functions and Eventarc

1. **Eventarc:** *The bridge service that automatically created a **Push Subscription** on your Pub/Sub topic to deliver messages to your **Cloud Function**.*
 - a. **Base64 Encoding:** Pub/Sub messages are sent in Base64 format (binary data encoded as text). Your function code automatically decoded it back to readable text.
2. **Decoupling:** Notice we didn't connect the publisher directly to the function.
 - a. The Pub/Sub topic sits in the middle. This means:
 - i. *The publisher can send messages even if the function is down or updating*
 - ii. *You could add more functions listening to the same topic*
 - iii. *Components can evolve independently*

Dataflow: Stream and Batch Data Processing Made Easy

1. **Dataflow** is Google Cloud's fully managed, serverless service for *processing large amounts of data*.
2. **Templates**: Use Dataflow Templates (Flex or Classic) to let non-technical users launch jobs from the Console or to automate deployments.
3. **Windowing**: Used to group **streaming data** into manageable chunks based on time.
4. Comparison:
 - a. **Dataproc** = Existing Hadoop/Spark workloads (Cluster-based).
 - b. **Dataflow** = New pipelines, Serverless, Apache Beam.

Reviewing Dataflow Job Status and Troubleshooting

1. **Job Graph:** Use the **visual graph** to identify exactly which step failed (look for the red box).
2. **Quotas:** If a job is stuck in **Queued**, check your project's CPU or IP address quotas.
3. **Logs:** Use **Worker Logs** to debug your specific code errors.
4. **Data Freshness:** *The primary metric for streaming health. It shows how far behind your pipeline is.*
5. **Drain vs. Cancel:**
 - a. **Drain**: *Graceful stop. No data loss. Use for production.*
 - b. **Cancel**: *Hard stop. Data loss. Use for dev.*
 - c. **Force Cancel**: *Use only if a job is stuck cancelling.*

Managed Service for Apache Kafka: Enterprise Event Streaming

1. **Migration Strategy:** Use **Managed Service for Apache Kafka** for "Lift and Shift" migrations of existing Kafka workloads to avoid rewriting code.
2. **Architecture:** It uses the modern **KRaft mode**, eliminating the need for ZooKeeper management.
3. **Tiered Storage:** It automatically offloads older data to **Google Cloud Storage** to lower costs while keeping recent data on **fast SSDs**.
4. **Comparison:**
 - a. **Pub/Sub:** Global, Serverless. Best for new Google Cloud applications.
 - b. **Kafka:** Regional, Open-Source compatible. Best for existing workloads and portability.

Section 12: Big Data & Analytics Platforms

Understanding File Storage vs. Object Storage: Use Cases

1. **Cloud Storage (Object):** Best for *immutable data* (write once, read many), storing backups, and serving web content. Accessed via APIs.
2. **Filestore (File):** Best for *migrating legacy applications* ("Lift and Shift") that **require a filesystem interface**. Accessed via **NFS** mount.
3. **Access Method:** If a question mentions "mounting" a volume to multiple Linux VMs, the answer is Filestore.

Filestore: Managed NFS for VM and GKE Workloads

1. **Filestore** is a **fully managed NFS** (Network File System) **server**.
2. **Protocol**: Uses NFS (v3/v4.1). It is the standard for shared Linux storage.
3. **Connectivity**: *Requires a Private IP in your VPC.*
 - a. *It is not internet-facing.*
4. **Security**: *Use IP-based rules for mounting access, and IAM for administrative management.*
5. **Availability**:
 - a. *Use **Zonal/Basic** for low cost (Dev/Test).*
 - b. *Use **Regional/Enterprise** for High Availability (Production).*

Hands-On: Creating a Filestore Instance and Mounting It

1. **Provisioning**: We created a Basic HDD instance with a 1 TiB capacity.
2. **Preparation**: We prepared the Linux client by installing **nfs-common** and creating a directory to serve as the mount point.
3. **Mounting**: We connected the storage using the standard Linux **mount** command directed at the instance's Private IP.
4. **Cleanup**: Finally, we deleted the resource to prevent billing for the allocated storage.

Enterprise NAS in the Cloud: An Overview of Google Cloud NetApp Volumes

1. **Google Cloud NetApp Volumes** is a fully-managed, cloud-based data storage service
2. **Protocols:** Supports...
 - a. **NFS** (Server Message Block) (Linux)
 - b. **SMB** (Network File System) (Windows)
3. **Best For:** *Windows workloads, Active Directory integration, and high-performance enterprise applications (SAP).*
4. **Management:** Fully managed service integrated into the Google Cloud Console.
5. **Service Levels:** *Flex, Standard, Premium, and Extreme (choose based on performance needs).*

Section 13: Virtual Private Cloud (VPC) Networking Fundamentals

What is a VPC? Your Slice of the Google Network

1. **VPC Definition:** A Virtual Private Cloud is a **logically isolated, global network** within your project.
2. **Scope:** VPC resources are Global.
 - a. They span all regions.
3. **Communication:** *Resources in the same VPC can communicate via internal IP addresses, even if they are in different regions.*
4. **Isolation:** *VPCs are isolated from each other.*
 - a. *Traffic does not flow between two different VPCs unless you explicitly connect them (using VPC Peering or VPN).*
5. **Modes:** *Always prefer **Custom Mode VPCs** for production environments to have full control over IP ranges.*

Understanding VPCs, Subnets, and IP Address Ranges

1. **Subnet Scope:** Subnets are Regional resources.
 - a. They define a range of IP addresses within a region.
2. **Cross-Zone:** A single subnet spans all zones within its region.
3. **CIDR Notation:** The suffix (like /24) determines the size.
 - a. *Lower number = more IPs.*
 - b. *Higher number = fewer IPs.*
4. **Expansion:** You can expand a subnet's IP range after creation, but you cannot shrink it.
5. **Secondary Ranges:** Used primarily for **Alias IPs** and GKE (Kubernetes) pods.

Hands-On: Creating a Custom Mode VPC with Subnets

1. **Custom Mode:** Always use Custom Mode to avoid creating unnecessary subnets in regions you don't use.
2. **Non-Overlapping:** Subnet IP ranges within the same VPC (or peered VPCs) must not overlap.
3. **Regional Creation:** You explicitly choose the region for each subnet during creation.
4. **Default Firewalls:** A custom VPC starts with zero firewall rules (blocks everything) unless you select the default convenience rules during creation.

Expanding Your Network: Adding Subnets and Expanding IP Ranges

1. **Flexibility:** *You can add new subnets to a Custom VPC at any time, in any region.*
2. **Expansion:** *You can increase the size of a subnet's IP range (expand the CIDR) after creation.*
3. **No Downtime:** *Expanding a subnet does not affect running workloads.*
4. **One Way:** *You cannot shrink a subnet's primary range.*
5. **Constraint:** *The new, larger range must not overlap with other subnets.*

Hands-On: Expanding a Subnet's IP Range

1. **Action:** *To expand a subnet, go to Subnet Details -> **Edit**.*
2. **Mechanism:** *Decrease the CIDR suffix (e.g., /24 to /20) to increase the number of IPs.*
3. That was the **fastest** recap! :)

Controlling Traffic with Cloud Next Generation Firewall (Cloud NGFW)

1. **Distributed:** Cloud NGFW is software-defined and **distributed**;
 - a. it is not a bottleneck appliance.
2. **Default Behavior:** Deny all **Ingress** (incoming), Allow all **Egress** (outgoing).
3. **Priority:** Lower integers indicate higher priority.
 - a. Rule 100 wins over Rule 1000.
4. **Stateful:** Return traffic for established connections is automatically allowed.
5. **Targets:** Use Network Tags or Service Accounts to target specific VMs, rather than opening ports to the whole network.

Hands-On: Creating Ingress and Egress Firewall Rules

1. **0.0.0.0/0:** This CIDR block represents the entire internet (any IP).
2. **Priority Wins:** *If you have an implied rule allowing traffic, but you create a specific Deny rule with a higher priority (lower number), the Deny rule wins.*
3. **Targets:** "All instances" is easy, but **dangerous**.

Advanced Firewalling with Tags and Service Accounts

1. **Network Tags:** Simple strings. Easy but less secure.
 - a. *Used in classic VPC firewall rules.*
2. **Service Accounts:** Identity-based. More secure.
 - a. *The preferred method for identifying targets in firewall rules.*
3. **Secure Tags:** IAM-governed tags used in network firewall policies (global/regional) and hierarchical firewall policies. Useful for organization-wide control.
4. **Targets:** You **CANNOT** use both network tags and service accounts in the same VPC firewall rule. Choose one or the other.

IP Addresses in GCP: Internal, External, and Static IPs

1. **Default Behavior:** VMs get an Ephemeral Internal IP by default.
 - a. When creating VMs through the console, they also get an Ephemeral External IP by default unless you explicitly configure the VM without one.
2. **Stopping VMs:** Stopping a VM releases its Ephemeral External IP.
 - a. It preserves its Internal IP.
3. **Cost:** *Static IPs cost money if they are reserved but not attached to a running resource.*
 - a. *Google charges you for "wasting" the IP. Internal static IPs have no cost.*
4. **Promotion:** You can "promote" an ephemeral IP to a static IP in the console if you decide you want to keep it.

Directing Your Traffic: Creating Custom Static Routes

1. **Priority:** Routes use a two-step evaluation.
 - a. **First**, the most specific destination always wins (e.g., /24 beats /16 regardless of priority numbers).
 - b. **Second**, if two routes have equally specific destinations, then priority decides— (reminder) lower numbers mean **HIGHER** priority. Think of it like a race: Priority 1 wins over Priority 100.
2. **Scope:** Routes apply to the entire VPC.
 - a. You cannot make a route that only exists in one subnet (unless you use advanced Policy Based Routing, but for ACE, assume global).
3. **Tags:** You can apply routes to specific VMs using Network Tags.
 - a. For example, "Only tagged 'vpn-users' use the VPN route."

Section 14: Connecting Your Networks

Connecting Two VPCs with VPC Network Peering

1. **VPC Peering Definition:** *A private connection between two VPC networks that allows communication using internal IP addresses.*
2. **Not Transitive:** *Peering does not flow through an intermediate VPC. You must create direct peering connections.*
3. **No Overlapping IPs:** **You cannot peer two VPCs if their IP ranges overlap.**
4. **Cross-Project and Cross-Org:** **Peering works across different projects and even different organizations.**
5. **Automatic Route Exchange:** *Routes are shared automatically, but firewall rules are not.*

Centralizing Your Network with Shared VPC

1. **Shared VPC Definition:** *Allows multiple projects to share subnets from a single VPC network.*
2. **Host Project:** **Owens the VPC. Managed by the networking team.**
3. **Service Projects:** *Use the VPC to deploy resources, but don't control the network.*
4. **IAM:** *Shared VPC Admin enables the setup. Network User role grants access to specific subnets.*
5. **Use Case:** Centralized network management across multiple projects in an organization.

Connecting to On-Premises: Cloud VPN Explained

1. **Cloud VPN Definition:** *Encrypted IPsec tunnel connecting on-premises network to your VPC.*
2. **HA VPN:** *99.99% SLA with two tunnels for redundancy.*
 - a. *Always prefer this over **Classic VPN**.*
3. **Cloud Router:** *Enables dynamic routing using BGP to automatically exchange routes.*
4. **Use Case:** *Hybrid cloud connectivity for **secure, cost-effective connection** to **on-premises**.*
5. **VPN vs. Interconnect:**
 - a. ***VPN** for moderate bandwidth.*
 - b. ***Interconnect** for high bandwidth with dedicated connections.*

Private Access to Google Services: Private Google Access

1. **Private Google Access Definition:** Allows VMs with only internal IP addresses to reach Google APIs and services.
2. **Scope:** Enabled **per subnet**, not per VPC or per VM.
3. **What It Provides:** Access to Google services like Cloud Storage, BigQuery, and Pub/Sub.
4. **What It Does NOT Provide:** Access to the general internet or third-party services.
5. **vs. Cloud NAT:** Private Google Access is for Google services only. **Cloud NAT** is for internet access.

Enabling Outbound Connections for Private VMs with Cloud NAT

1. **Cloud NAT Definition:** Provides **N**etwork **A**ddress **T**ranslation for VMs without external IPs to access the internet.
2. **Outbound Only:** VMs can initiate outbound connections, but cannot receive inbound traffic from the internet.
3. **Regional Scope:** Cloud NAT is configured per region and requires **Cloud Router**.
4. **Security Benefit:** VMs stay private (no external IPs), reducing attack surface.

Managing Domain Names with Cloud DNS

1. **Cloud DNS Definition:** Google's managed DNS service that ***translates*** domain names to IP addresses.
2. **Public Zones:** For domain names ***accessible to anyone on the internet***.
3. **Private Zones:** For ***internal domain names*** only resolvable within your VPC.
4. **Record Types:**
 - a. *A records (IPv4)*
 - b. *CNAME (aliases)*
 - c. *MX (mail servers)*
 - d. *TXT (verification)*
5. **Integration:** Works seamlessly with ***load balancers*** and other ***Google Cloud resources***.

Section 15: Load Balancing and Network Tiers

Why Do We Need Load Balancing?

1. **Traffic Distribution:** Load Balancers distribute user traffic across multiple instances to prevent overloading a single server.
2. **Single Entry Point:** They provide a single frontend IP address (single anycast IP address) for your application, hiding the complexity of your backend infrastructure.
3. **Health Checks:** They continuously monitor backend instances and automatically remove unhealthy ones from the pool to ensure High Availability.
4. **SSL Termination:** They can handle encryption and decryption, reducing the CPU load on your backend instances.
5. **Seamless Scaling:** They work perfectly with Autoscaling to add or remove backends without disrupting user connections.

The GCP Load Balancer Family: A High-Level Overview

1. **HTTP/HTTPS Traffic?** Use an Application Load Balancer (Layer 7).
 - a. It enables smart routing based on URLs or headers.
2. **TCP/UDP Traffic?** Use a Network Load Balancer (Layer 4).
3. **Need Client IP Preservation?** Use a Passthrough Network Load Balancer.
 - a. It preserves the source IP and uses Direct Server Return.
4. **Internal Traffic Only?** Use an **Internal Load Balancer** to keep traffic private and secure within your VPC.
5. **Global Reach?** Application Load Balancers are typically **Global**, allowing a single Anycast IP for the whole world (more on this in the next lesson).

Global vs. Regional Load Balancing

1. **Scope:** Global Load Balancers use a single Anycast IP worldwide. Regional Load Balancers use an IP specific to one region
2. **Failover:** Global Load Balancers provide Cross-Region Failover. If one region fails, traffic automatically moves to another region. Regional Load Balancers cannot do this; if the region is down, the service is down
3. **Performance:** Global Load Balancers offer lower latency by entering Google's network at the closest Edge Location (Premium Tier)
4. **Compliance:** Use Regional Load Balancers when you have **strict data residency requirements**. Regional load balancers ensure that TLS termination, traffic processing, and all backends remain within a specific geographic region
5. **Exam Tip:** If a question asks for "High Availability across multiple regions" or "Single global IP," choose a Global Load Balancer. If it mentions "Regulatory compliance" or "Data must not leave the region," choose Regional.

Hands-On: Setting Up a Global External HTTPS Load Balancer

1. **Frontend:** We created a Global Anycast IP that listens for traffic from the internet.
2. **Backend Service:** We grouped our US and EU resources into a **single logical service**.
3. **Health Check:** We configured a probe to ensure traffic only goes to healthy instances.
4. **Routing Rules:** We told the Load Balancer to send all traffic to our backend service.
5. **Firewall Note:** *In production, you would create a firewall rule allowing only Google's health check IP ranges (130.211.0.0/22 and 35.191.0.0/16) on your health check ports.*
 - a. For this demo, our firewall rule allows broader HTTP access for simplicity.

Understanding Network Service Tiers: Premium vs. Standard

1. **Premium Tier:** Uses Google's global private backbone ("**Cold Potato**").
 - a. It delivers **high performance** and is required for Global Load Balancing.
2. **Standard Tier:** Uses the public internet ("**Hot Potato**").
 - a. It is **lower cost** but has **variable performance**.
 - b. It only supports **Regional Load Balancers**.
3. **The Exam Rule:** If the question prioritizes speed or global reach, choose **Premium**.
 - a. If the question prioritizes cost reduction for a regional app, choose **Standard**.

Section 16: Infrastructure as Code (IaC) with Terraform

What is Infrastructure as Code (IaC)?

1. **Definition:** *IaC is the management of infrastructure (networks, VMs, load balancers) using code and configuration files instead of manual processes.*
2. **Declarative Model:** *You define the desired state (what you want), and the tool ensures the environment matches that state.*
3. **Version Control:** *IaC files should be stored in source control systems (like **Git**) to track history and enable collaboration.*
4. **Reproducibility:** *IaC ensures that development, staging, and production environments are consistent and identical.*
5. **Automation:** ***Reduces** human error and increases the speed of deployment.*

IaC Tooling in GCP: Terraform, Config Connector, and Fabric FAST

1. **Terraform:** The most common open-source IaC tool.
 - a. It uses the Google Cloud Provider to interact with APIs.
 - b. It creates, updates, and destroys resources based on configuration files.
2. **Config Connector:** A Kubernetes extension.
 - a. It allows you to manage Google Cloud resources (like Cloud SQL or Pub/Sub) using Kubernetes-style YAML manifests and **kubectl**.
3. **Fabric FAST:** An open-source framework provided by Google.
 - a. *It delivers a production-ready "landing zone" (resource hierarchy, networking, security) powered by Terraform modules.*
4. **Declarative Nature:** All three tools follow the declarative model—you define the end state, and the tool ensures the cloud matches it.

Hands-On: Deploying VM with Terraform

1. **main.tf:** The primary configuration file where you define your provider and resources.
2. **Provider Block:** Configures the cloud platform (e.g., provider "google").
3. **Resource Block:** Defines a specific infrastructure object (e.g., resource "google_compute_instance").
4. **terraform init:** Initializes the directory and downloads necessary provider plugins.
5. **terraform plan:** Creates an execution plan, previewing changes without applying them.
6. **terraform apply:** Executes the plan and provisions the resources in the cloud.
7. **terraform destroy:** Removes all resources defined in the configuration file.

Terraform State & Updates

1. **terraform.tfstate**: A JSON file that maps your Terraform configuration to real-world resource IDs.
2. **Local State**: Stored on the machine running Terraform. **Not suitable for teams.**
3. **Remote State**: The best practice for production. Stores the state file in an external system like Google Cloud Storage (GCS).
4. **State Locking**: A native feature of GCS backends that prevents multiple users from modifying infrastructure simultaneously through automatic lock file creation.
5. **Automatic Refresh**: Terraform automatically updates its state with the real-world status during plan and apply operations.
 - a. For manual state updates, use **terraform plan -refresh-only** or **terraform apply -refresh-only** (the standalone **terraform refresh** command was deprecated in v0.15.4).
6. **Destructive Updates**: Changes that require deleting and recreating a resource (e.g., changing a resource's region).

Section 17: Monitoring with Cloud Monitoring

The Google Cloud Observability Suite

1. **Google Cloud Observability** is the current name for the suite of tools formerly known as **Stackdriver**.
2. **Cloud Monitoring** tracks the **performance** and **health metrics** (the "vitals") of your resources.
3. **Cloud Logging** collects and stores **text-based records (logs)** of events and operations.
4. **Cloud Trace** is used to **identify performance bottlenecks** and **latency** in distributed applications.
5. **Error Reporting** aggregates **application crashes** into a clear dashboard for quick triage.
6. **Cloud Profiler** analyzes **code performance** to help optimize resource consumption (CPU/Memory).

Cloud Monitoring 101: Metrics, Dashboards, and Uptime Checks

1. **Metrics** are numerical measurements collected over time (like CPU usage or disk space).
2. **System Metrics** are collected automatically by Google Cloud services without extra configuration.
3. **Metrics Explorer** is a tool for **temporary**, ad-hoc **querying** and visualization of metrics.
4. **Dashboards** are **permanent collections of charts** used for continuous monitoring of your systems.
5. **Uptime Checks** **verify the availability of your service** by accessing it from external locations around the world.

Hands-On: Exploring Metrics for a Compute Engine VM

1. **Observability Tab:** For a single VM, the easiest way to view metrics is to go to the Compute Engine page, click the VM name, and select the "**Observability**" tab.
2. **Metrics Explorer:** For advanced analysis, navigate to *Monitoring > Metrics Explorer*.
3. **Metric Selection:** *You select metrics by choosing the **Resource** (e.g., VM Instance) and the **Metric Name** (e.g., CPU utilization).*
4. **Filtering & Grouping:** *You can organize your data by grouping (e.g., by zone) or filtering (e.g., by instance name) to find exactly what you need.*

Creating Alerts: Getting Notified When Things Go Wrong

1. **Alerting Policies** allow you to monitor your infrastructure proactively without watching dashboards 24/7.
2. **Conditions** define the threshold (e.g., > 80%) that determines when an alert should fire.
3. **Retest Window** helps prevent false alarms by requiring the condition to be true **for a set period** (e.g., 5 minutes) before triggering.
4. **Notification Channels** determine where the alert is sent, such as Email, SMS, Slack, or PagerDuty.
5. **Incidents** are records created in the console when an alert fires, **helping you track active issues**.

Hands-On: Creating a CPU Utilization Alert

1. **Create Policy:** Alerts are created in the **Monitoring > Alerting** section.
2. **Select Metric:** You must define exactly which resource and metric to watch, such as **VM Instance > Instance > CPU utilization**.
3. **Threshold:** The specific value (e.g., 80%) that the metric must cross to be considered "bad".
4. **Incident Autoclose:** A safety timer that automatically closes an alert if the data disappears for a set period (defaulting to 7 days).
5. **Documentation:** Custom instructions you add to the alert body to help the recipient troubleshoot and fix the problem faster.

Going Deeper with User-defined metrics

1. **User-defined metrics** allow you to track **application-specific** or **business-level data** that Google Cloud cannot measure automatically (e.g., "sales count" vs. "CPU usage").
2. **The Ops Agent** can be configured to collect metrics from third-party applications like Nginx or PostgreSQL running on your VMs.
3. **Managed Service for Prometheus (GMP)** is the recommended way to collect metrics from **Kubernetes clusters**.
4. **Logs-based Metrics** allow you to convert text log entries into numerical metrics (e.g., counting error messages).
5. **Cost**: unlike standard system metrics, User-defined metrics are usually billable based on the volume of data ingested.

Your Personal Health Advisor: The Service Health Dashboard

1. **"Is it me or Google?":** The **Personalized Service Health dashboard** is your first stop to determine if a problem is caused by your configuration or the underlying Google platform.
2. **Personalized View:** *Unlike the public Google Cloud Service Health page, Personalized Service Health only shows events relevant to your specific active projects and regions.*
3. **Event States:** *Shows Active incidents (including Emerging and Confirmed states) and Closed incidents (Resolved, Merged, etc.).*
4. **Relevance Levels:** Filters incidents by how they impact you - Impacted, Related, Partially Related, or Not Impacted.
5. **Alerting:** You can configure alerts to be **notified automatically** when Google posts a new incident relevant to you.
6. **Mobile Access:** Available through the Google Cloud **mobile app** for on-the-go monitoring.

Let AI Help: Using Gemini Cloud Assist for Monitoring

1. **Gemini Cloud Assist** is an **AI collaborator integrated directly into the Google Cloud Console**.
 - a. **Explain Log Entries:** You can select a cryptic log message in Cloud Logging and click *"Explain this log entry"* to get plain language explanations and suggested fixes.
 - b. **Natural Language Chat:** You can ask questions using the **Gemini chat panel** about resource performance, Google Cloud best practices, and get help generating gcloud commands.
 - c. **Investigations (Public Preview): Gemini can correlate data across logs, metrics, configurations, and Cloud Asset Inventory** to produce observations and hypotheses about probable root causes with recommended next steps.
 - i. You can also transfer investigations to Google Cloud support cases.

Section 18: Introduction to Cloud Logging: The Central Log Hub

Introduction to Cloud Logging: The Central Log Hub

1. **Cloud Logging** is a fully managed service for real-time log management and analysis.
2. **The Log Router** controls the flow of logs.
 - a. It checks every log entry against **rules** to decide where it should go.
3. **Sinks** are the configuration objects that define where logs are routed (e.g., to a bucket, BigQuery, or Pub/Sub).
4. **Log Buckets** are **containers** inside Cloud Logging where logs are stored.
 - a. The **_Required bucket** stores critical audit logs for 400 days and cannot be disabled.
 - b. The **_Default bucket** stores other logs for 30 days by default, but this retention can be customized.

Viewing and Filtering Logs in the Logs Explorer

1. The **Logs Explorer** is the primary tool for searching and filtering logs.
2. **The Timeline** helps you visualize log volume and spot anomalies over time.
3. **textPayload** is used for simple unstructured logs, while **protoPayload** or **jsonPayload** holds structured data like audit trails.
4. You can create filters quickly by clicking on specific field values and selecting "*Show matching entries.*"
5. Always remember to click **Run Query** to update your view after changing filters.

Understanding Log Sinks: Exporting Logs for Analysis

1. **Log Sinks** allow you to **export logs continuously** to other services.
2. **Cloud Storage** is the **best destination for low-cost**, long-term **retention** and **compliance**.
3. **BigQuery** is the destination for **performing SQL-based analytics** on your log data.
4. **Pub/Sub** is used to stream logs to **third-party tools** or external applications.
5. **Aggregated Sinks** can be set at the **Organization** or **Folder** level to export logs from all child projects centrally.
6. **Sinks only export new logs that arrive after the sink is created;**
 - a. they do not retroactively export old data.

Hands-On: Creating a Log Sink to a Cloud Storage Bucket

1. **Log Router** is where you manage Sinks.
2. **Sinks** require a **Destination** (like a Bucket) and a **Filter** (like **severity >= ERROR**).
3. Google Cloud handles the IAM permissions for the sink **automatically** when you create it in the console (it grants the sink's identity write access to the bucket).
4. **Latency exists:** Logs sent to Cloud Storage are batched, so they do not appear instantly. If you need real-time analysis, you would use Pub/Sub or BigQuery instead.

Configuring Log Buckets and Log Analytics

1. **Log Buckets** are containers **within** Cloud Logging that store your log data.
2. **Buckets are Regional.**
 - a. This is critical for data residency requirements (GDPR).
3. **Retention is configurable.**
 - a. You can increase retention from the default 30 days up to 3650 days (10 years)
4. **Log Analytics** allows you to run SQL queries directly against your log data **without** exporting it to BigQuery first
5. You can **upgrade** an existing bucket to use Log Analytics.
6. This simplifies your architecture by removing the need for complex export sinks when you just want to run analytics.

Understanding and Configuring Audit Logs

1. **Cloud Audit Logs** answer *"Who, What, Where, and When."*
 - a. **Admin Activity Logs** track configuration changes (e.g., creating a VM).
 - i. They are **always on, free**, and stored for 400 days.
 - b. **Data Access Logs** track access to data (e.g., reading a file).
 - i. They are **disabled by default** (except for BigQuery) and **chargeable**.
2. You configure **Data Access logs** in the **IAM & Admin console**, not in the Logging console.
 - a. **System Event Logs** track actions taken by Google systems.
 - b. **Policy Denied Logs** record when access is blocked by security perimeters.

The Ops Agent: Better Metrics and Logs from Your VMs

1. **Standard Metrics** (CPU, Network, Disk I/O) are available automatically **without any agent**.
2. **Memory (RAM) Utilization** and **Disk Space** usage require the **Ops Agent** because they are **internal** to the Guest OS.
3. **The Ops Agent** collects both **Logs** and **Metrics**, replacing the legacy Stackdriver agents.
4. It supports Third-party application integration, allowing you to scrape metrics and logs from software like **Nginx**, **Cassandra**, and **SQL Server**.
5. You can manage installation at scale using **VM Manager** or **Agent Policies**.

Diagnosing Application & Database Issues

1. **Cloud Trace** is for **distributed tracing**.
 - a. Use it to find **latency** bottlenecks in microservices (the "Waterfall" view)
2. **Cloud Profiler** is for **code performance**.
 - a. Use it to find CPU or Memory leaks in specific functions
 - b. It uses statistical sampling to ensure it adds almost no overhead to production apps.
3. **Query Insights** helps you **identify slow SQL queries** in Cloud SQL based on CPU and latency.
4. **Index Advisor** analyzes **database usage** and **recommends new indexes** to improve performance.

Proactive Optimization with Active Assist

1. **Active Assist** uses machine learning to provide actionable recommendations.
2. **Idle VM Recommender** identifies virtual machines that are running but not being used, helping you reduce waste
3. **Rightsizing** suggests changing machine types (e.g., from n1-standard-4 to n1-standard-1) based on actual resource utilization.
4. **IAM Recommender** helps enforce the Principle of Least Privilege by identifying users with "Over-privileged" access (roles they don't use)
5. **Unattended Project Recommender** helps clean up abandoned projects.
6. Recommendations can be applied directly from the console or exported for analysis.